

절차 그래프의 이해

LLM 에이전트를 위한 자기진화 실행 구조 — Procedural Graphs 한국어판

- 문제 → 프레임워크 → 실험 → 진화 → 효율성 → 실무 시사점
- 절차 그래프가 무엇을 명시화하고, 어떻게 진화하며, 어디서 이득이 되는지
- arXiv 2609.09153 논문 정독서 한국어판

차례

들어가며	3
01 1. 왜 절차 그래프인가	5
02 2. 관련 연구	8
03 3. 절차 그래프 프레임워크	12
04 4. 실험 설계	18
05 5. 주요 결과	21
06 6. 장기 의사결정과 생존	24
07 7. 그래프 구축 전략	28
08 8. 자기진화 10라운드	31
09 9. 효율성 분석	35
10 10. 결론과 실무 시사점	38
부록 — 프롬프트와 알고리즘	40

DAY

들어가며



언어 모델은 이제 계획을 세우고 외부 도구를 실제로 다루는 에이전트 형태로 실무에 배치된다. 그런데 이런 에이전트가 무너지는 지점은 지식이 부족해서가 아니다. 무엇을 해야 하는지, 어떤 순서로 해야 하는지, 어떤 조건에서 해야 하는지라는 절차 지식이 모델 안에 암묵적으로만 존재하기 때문이다. 대부분의 에이전트는 지금까지의 행동과 관찰이 쌓인 기록 위에서 자유로운 생성으로 다음 행동을 고르고, 절차의 일관성을 지키는 일 전체를 이 자유 생성에 떠넘긴다. 궤적이 길어지면 목표를 잃고, 도구를 엉긴 순서로 부르고, 생산성 없는 행동을 되풀이한다. 이 책은 바로 그 지점을 다룬다.

“원문: Procedural Graphs: Self-Evolving Execution Structures for LLM Agents — arXiv:2609.09153v1 (2026-09) · Yuxing Lu, Yicheng Chen, Shanchan Wu, Sercan Ö. Arık (Google)”

“링크: <https://arxiv.org/abs/2609.09153>”

논문이 내놓은 답은 절차 그래프(Procedural Graph, PG)라는 표현이다. 지식 그래프가 세계의 사실을 (entity, relation, entity) 삼중조로 정리해 무엇이 무엇인지 묻는 질문에 답하듯, 절차 그래프는 절차 지식을 (procedure, relation, procedure) 삼중조로 정리해 무엇을 해야 하는지 묻는 질문에 답한다. 노드는 도구 행동과 추론 단계, 상태를 추상화하고, 에지는 허용되는 전이와 그 전이를 언제 어떻게 취해야 하는지를 적어 둔다. 절차 지식이 모델 가중치 밖의 명시적 구조로 나오기 때문에 검사하고, 조희하고, 재학습 없이 고칠 수 있다.

이 그래프는 두 국면에서 움직인다. 온라인 추론 국면에서 프레임워크는 에이전트 궤적에서 지금 서 있는 활성 노드의 위치를 좁혀 찾고, 가이던스 모델이 주변 서브그래프를 단계 수준의 상황 지침으로 번역한다. 이 지침은 다음 행동을 지시하지 않고 방향을 기울일 뿐이므로 솔버의 추론 자유는 그대로 남는다. 오프라인 자기진화 국면에서는 LLM 정제기가 실패한 궤적과 성공한 궤적을 대조해 그래프의 위상과 속성을 편집하고, 홀드아웃 검증 성능을 지키거나 끌어올린 편집만 채택한다. 거부된 후보는 거부 메모리에 남아 같은 제안의 반복을 막는다. 최소 뼈대에서 출발해도 이 순환이 손으로 설계한 그래프에 필적하거나 능가하는 그래프를 만들고, 결함이 있는 전문가 사전을 고칠 수도 있다.

이 책은 원문의 논리를 따라 열두 개의 장으로 짜였다.

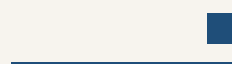
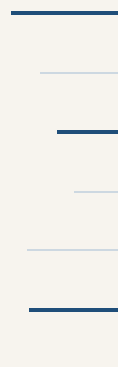
- 도입부에서 오늘날 에이전트의 실패 양상과 기존 접근의 한계를 짚고, 절차 그래프가 왜 필요한지 살펴본다.
- 중반부에서 절차 그래프의 표현과 ‘프레임워크’ 장의 설계, 관련 연구와의 위치 관계를 다룬다.
- 실험 부분에서 다양한 과업과 LLM 위의 측정 결과, 생성형 가이던스와 자기진화가 각각 얼마나 기여하는지 확인한다.
- 마무리 부분에서 자기진화 루프의 작동을 자세히 뜯어보고, 실무 적용과 한계, 앞으로의 방향을 정리한다.

에이전트 시스템을 설계하고 운영하는 개발자라면 이 책에서 두 가지를 얻을 것이다. 하나는 프롬프트와 메모리 설계만으로 흉내 내기 어려웠던 절차 지식을 명시적 구조로 꺼내 놓는 방법이고, 다른 하나는 그 구조를 실행 시점 가이던스와 실행 뒤 진화 순환으로 잇는 설계 감각이다. 모델을 바꾸거나 재학습하지 않고도 에이전트의 절차적 품질을 끌어올리고 싶은 독자에게 이 책이 출발점이 되기를 바란다.

DAY

01

1. 왜 절차 그래프인가



자유 생성에 떠넘겨진 절차 일관성

언어 모델(LLM)은 장기간에 걸친 계획을 세우고 외부 도구로 행동하는 자율 에이전트 형태로 배치 범위를 넓혀 왔다. 그런데 오늘날 대부분의 에이전트는 다음 행동을 고를 때 특별한 제약을 두지 않는다. 지금까지의 행동과 관찰이 평평하게 쌓인 기록을 조건으로 삼아, 자유로운 생성으로 행동을 뽑는 방식이다. 이 구조에서 절차의 일관성은 모델의 자유 생성에 전적으로 떠넘겨진다. 에이전트는 스스로 어떤 관찰이 관련 있는지 가려내고, 앞으로 어떤 단계가 남아 있는지 추론하고, 그 단계 사이의 의존 관계를 지키는 행동을 골라야 한다.

기록이 짧을 때는 이 부담을 감당할 수 있다. 문제는 궤적이 길어질 때다. 저자들은 이때 에이전트가 목표를 놓치고, 도구를 순서대로 부르지 못하고, 생산성 없는 행동을 되풀이한다고 진단한다. 세 실패 모두 지식이 없어서가 아니라, 무엇을 어떤 순서로 어떤 조건에서 해야 하는지가 어디에도 명시돼 있지 않기 때문에 생긴다. 쌓이는 기록은 과거에 무엇을 했지만 알려 줄 뿐, 앞으로 무엇이 허용되는지는 말해 주지 않는다.

기존 접근 세 가지

절차적 구조를 주려는 시도는 이미 여럿 있다. 텍스트 메모리와 자기성찰 계열(ReAct 계열 에이전트의 실패 기록을 활용하는 방법을 포함한다)은 지난 경험을 자유 형식의 텍스트로 적어 두었다가 필요할 때 꺼내 문맥에 넣어 재사용하게 한다. 기록에는 쓸모 있는 경험이 남지만, 그 경험이 지금 이 단계에 어떻게 적용되는지, 뒤따르는 단계를 어떻게 구속하는지는 매번 솔버가 스스로 재구성해야 한다. 같은 교훈을 상황이 바뀔 때마다 새로 해석하는 셈이다.

상태 조건형 가이드라인은 상태에 따라 골라 주는 규칙 조언이라서 더 겨냥된 도움을 준다. 다만 규칙을 검색해 줄 뿐, 절차 단계와 단계 사이를 잇지는 못한다. 어떤 규칙이 끝난 뒤 어떤 단계가 허용되는지는 여전히 모델의 몫으로 남는다.

워크플로와 상태 기계는 단계를 명시적으로 드러내고 실행을 구속하기 때문에 앞의 두 접근보다 구조가 단단하다. 그러나 그 단계를 사람이 손으로 설계해야 한다. 워크플로 구조를 오프라인에서 최적화하는 자동 탐색 연구는 이 설계 수고를 덜어 주지만, 저자들이 짚는 남은 과제는 그대로다. 편집할 수 있는 절차 표현에, 에이전트의 현재 진행 상황에 맞춰 조건이 붙는 가이드언스를 결합하는 것이다.

에이전트가 절차 지식에 바라는 네 가지

저자들은 에이전트에게 필요한 절차 지식의 조건을 네 가지로 정리한다.

- **구조화**: 무효한 행동에서 벗어나도록 방향을 잡아 줄 만큼 구조가 명시적이어야 한다.
- **유연성**: 구조가 추론의 자유를 지키며, 모델이 스스로 생각할 여지를 짓누르지 않아야 한다.
- **진행 상황 응답성**: 지금 어디쯤 와 있는지에 따라 도움의 내용이 달라져야 한다.
- **경험으로 개선**: 실행을 거듭하면서 함께 나아져야 한다.

절차 그래프: what-to-do를 위한 삼중조

저자들은 이 네 조건을 절차 그래프(Procedural Graph, PG)로 충족시킨다. 절차 그래프는 절차 지식을 담은 명시적이고 편집 가능한 방향 그래프다. 설계를 보면 익숙한 구조가 그대로 옮겨 왔다. 지식 그래프가 사실 지식을 (entity, relation, entity) 삼중조로 정리해 what-is 질문에 답하듯, 절차 그래프는 절차 정보를 (procedure, relation, procedure) 삼중조로 정리해 what-to-do 질문에 답한다. 노드는 도구 행동과 추론 단계, 상태를 추상화한다. 에지는 허용되는 전이를 인코딩하고, 그 전이를 어떻게 언제 취해야 하는지를 설명하는 텍스트 속성을 단다.

절차 지식이 모델 가중치 밖의 구조로 존재한다는 점이 이 설계의 요체다. 가중치 안에 묻혀 있지 않기 때문에 사람이 눈으로 검사할 수 있고, 실행 중 매 단계 조회할 수 있고, 재학습 없이 고칠 수 있다. 프롬프트 몇 줄에 압축하는 방식과 달리, 지식의 양이 문맥 창을 갹아 먹지도 않는다.

그림 1-1은 두 그래프의 역할 차이를 한 화면에 나란히 보여준다. 왼쪽의 지식 그래프는 “모나리자는 어디에 있나”라는 질문을 받는다. 모나리자가 다빈치가 그렸고(painted_by), 그림이라는 개념에 속하며(is_a), 루브르에 있다는(is_in) 사실이 삼중조로 놓여 있어, 질문에 대한 답은 “루브르, 파리”다. 오른쪽의 절차 그래프는 같은 상식을 쓰되 물음이 다르다. “에이전트가 다음에 무엇을 해야 하나”라는 물음 앞에 검색하고, 근거를 읽고, 필요한 값을 꺼내고, 계산하고, 답을 검사한 뒤 출력으로 마치는 일련의 절차가 노드와 전이로 놓여 있다. 전이는 어떤 단계가 무엇을 가능하게 하고(enables), 무엇을 끌어내고(extracts), 무엇을 요구하는지(requires)로 이름이 붙고, 그래프는 지금 단계에서 “다음은 답을 검사한 뒤 출력한다”라고 가리킨다. 사실의 모음이 무엇이 무엇인지 묻는 질문에 지식을 주는 구조라면, 절차의 모음은 지금 무엇을 해야 하는지 묻는 질문에 가이드를 주는 구조다.

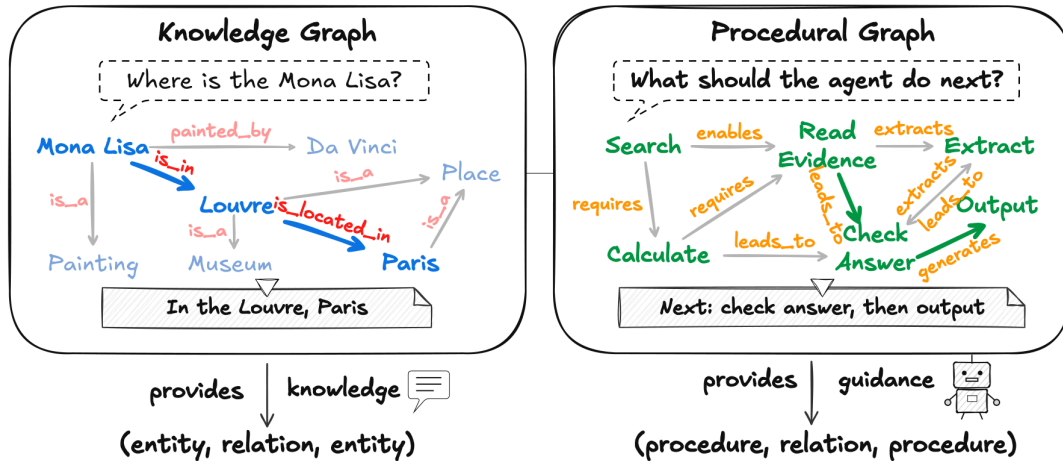


그림 1-1 지식 그래프와 절차 그래프 — 원문 그림 1을 재수록했다

두 국면으로 움직이는 프레임워크

저자들은 이 그래프를 서로 보완하는 두 국면에 배치한다. 온라인 추론 국면에서 프레임워크는 에이전트 궤적에서 활성 노드의 위치를 좁혀 찾고, 가이드선 모델이 주변 서브그래프를 위상적 맥락까지 포함해 읽은 뒤, 관련 에지 속성을 다음 단계의 상황 지침으로 번역한다. 오프라인 자기진화 국면에서는 학습 과업 묶음을 하나씩 마칠 때마다 LLM 정제기가 실패한 궤적과 성공한 궤적을 대조하며 그래프의 위상과 속성에 대한 편집을 제안한다. 빠진 노드와 에지를 더하고, 실패를 부르는 것을 잘라 내고, 에지 속성을 다듬는 작업이다. 구조상 문제가 없는 후보 그래프는 홀드아웃 검증 점수를 지키거나 끌어올릴 때 채택되고, 거부된 후보는 부정적 제약으로 남는다. 검증 게이트가 점수를 낮추는 후보를 걸러 내고, 거부 메모리가 같은 실패 제안의 반복을 막는다.

논문이 내세우는 기여

원문은 저자들의 기여를 네 가지로 요약한다.

- 절차 그래프(PG): 솔버의 추론 유연성을 지키면서 LLM 에이전트 실행을 이끄는, 명시적이고 편집 가능한 절차 지식 그래프를 제시한다.
- 생성형 PG 가이드선: 정적인 그래프와 살아 있는 궤적을 단계 수준의 상황 지침으로 바꾸는 온라인 메커니즘을 제안한다.
- 자기진화 루프: 실행 피드백으로 그래프의 위상과 속성을 다듬는 순환을 개발한다.
- 측정된 효과: 다양한 과업과 LLM에서 PG가 메모리 기반 베이스라인을 일관되게 앞선다. 백지에서의 진화는 손으로 설계한 그래프에 필적하거나 능가하는 그래프를 만들고, 같은 루프가 결함이 있는 전문가 사전도 고친다.

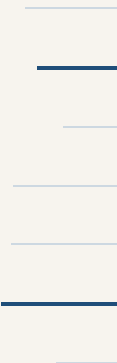
다음 장에서는 이 표현이 경쟁하는 기존 연구들을 자세히 살펴보고, 절차 그래프가 어디에 자리 잡는지 확인한다.

참고: 원문 §1 (pp.1-2)

DAY

02

2. 관련 연구



절차 그래프(Procedural Graph, PG)를 제 위치에 놓으려면 세 갈래 기존 연구를 살펴봐야 한다. 행동 선택 방식, 계획에 쓸 구조를 미리 제공하는 방식, 궤적에서 재사용 지식을 끌어내는 방식이다. 각 계열이 무엇을 비워 두는지 짚은 뒤 부록의 CoALA 관점과 8차원 비교로 절차 그래프의 차이를 구체화한다.

2.1 LLM 에이전트와 행동 선택

오늘날 LLM 에이전트 실행의 지배적 틀은 자유 형식의 행동 선택 루프다. ReAct가 추론과 환경 행동을 번갈아 수행하는 구조를 제시한 뒤 후속 연구들은 이 틀을 넓혔다. Reflexion은 시행 사이에 자기비판을 끼워 넣고, Tree-of-Thoughts와 Graph-of-Thoughts는 사슬형 추론을 분기 탐색으로 일반화하며, Plan-and-Solve와 ADaPT는 계획과 실행을 분리한다. CodeAct는 행동을 실행 가능한 파이썬 코드로 통일한다. 도구 카탈로그를 키운 계열도 이어진다. ToolLLM은 16,464개 도구 벤치마크와 깊이 우선 결정 트리를 함께 제시했고, Gorilla와 AnyTool은 검색 구조를 다듬으며, ToolGen은 조회와 호출을 다음 토큰 생성에 통합한다.

이 방법들은 유효한 다음 행동을 문맥 정보만 보고 모델이 고르게 맡기며, 허용 전이는 암묵적으로 남는다. 계획 환각, 긴 과업에서의 궤적 이탈, 실행 피드백이 모호할 때의 반복 루프가 문서화된 대표적 실패 모드다. 절차 그래프는 이 암묵적으로 남은 전이를 명시적인 대상으로 만들려고 한다.

2.2 에이전트 계획을 위한 구조화된 사전

두 번째 계열은 계획에 쓸 명시적 구조를 미리 제공한다. KnowAgent는 허용 가능한 행동 규칙의 텍스트 지식베이스로 궤적 합성 동안 행동 경로를 제약한다. FlowBench는 워크플로 지식을 텍스트, 코드, 플로차트 세 형식으로 형식화하고, 여섯 도메인 51개 시나리오에서 플로차트가 계획 환각을 가장 효과적으로 줄임을 보였다. AFlow는 워크플로 구축을 코드로 표현된 그래프 위의 몬테카를로 트리 탐색으로 재정의한다. 디코딩 제약도 구조의 한 형태인데, TOOLDEC는 도구 문법 스키마를 유한 상태 오토마타로 컴파일해 문법적으로 유효한 호출만 생성하게 강제한다. 다만 보장되는 것은 표층 형식의 유효성까지이며, 과업 상태에서 그 행동이 의미적으로 허용되는지는 말해 주지 않는다.

ToolNet은 LLM이 만든 궤적에서 도구 전이 그래프를 채굴해 추론 시점에 그 위를 걷게 하고, ControlLLM은 매개변수 타입 매칭으로 도구 의존 그래프를 사전에 구축한다. GraphRAG-Tool Fusion은 수작업 도구 지식 그래프 위에서 벡터 조회와 그래프 순회를 결합해 GraphRAG를 도구 선택으로 확장한다.

이 그래프들은 도구 선택과 연산자 실행 등 저마다 다른 쓰임을 가지며, 조건부 행동 지식은 KnowAgent의 텍스트 규칙과 FlowBench의 분기 워크플로에 있다. 절차 그래프는 도구 호출과 추론 단계를 아우르는 그래프 위에 `condition guidance pitfalls` 속성을 붙인 전이를 엮고, 매 단계 현재 절차의 위치를 좁혀 찾아 주변 이웃을 언어화한다. AutoGuide도 상태에 맞는 가이드라인을 고르는 단계별 조회를 쓰지만 연결된 전이를 조회하고 토폴로지와 속성을 편집하는 기능은 없다.

2.3 궤적에서 자기 개선하는 에이전트

세 번째 계열은 과거 궤적에서 재사용 가능한 지식을 증류한다. Reflexion은 자기비판을 에피소드 버퍼에 적어 두고 과업을 다시 시도하고, MemoryBank는 망각 스케줄로 경험 요약을 관리하며, RAP은 과거 궤적 통째를 문맥 예시로 조회한다. 두 가지 방법이 대조적 증류로 나아간다. ExpeL은 성공과 실패 궤적을 짝지어 대조하며 자연어 인사이트를 뽑아 내고, AutoGuide는 이를 “문맥 X에서 행동 Y가 적절하다” 꼴의 조건부 가이드라인으로 다듬어 테스트 시점에 현재 상태에서 조회한다.

재사용할 기술과 워크플로를 저장하는 가족도 있다. Voyager는 자연어 서술을 임베딩으로 색인한 기술 라이브러리를 과업마다 top-k로 조회하고, AWM은 자연어 서술과 프로그램형 행동을 결합한 재사용 워크플로를 오프라인과 온라인 양쪽에서 프롬프트 메모리에 추가한다.

절차 기억의 수명주기를 명시적으로 다루는 틀도 나왔다. MemP는 구축, 조회, 갱신을 최적화 대상으로 형식화하고 궤적을 세밀한 단계 지침과 상위의 스크립트 추상으로 증류하며, EvolveR은 오프라인 자기 증류와 온라인 전략 원칙 조회로 고

리를 닫는다. A-Mem과 Zep/Graphiti는 지식 그래프식 구조를 에이전트 메모리에 도입하지만 초점은 절차가 아니라 에피소드와 의미 기억이다.

이 가운데 AutoGuide가 절차 그래프와 가장 가깝다. 에지 속성처럼 조건부 가이드라인이 상황과 행동을 잇기 때문이다. 절차 그래프는 이런 전이들을 타입 있는 그래프로 연결해 구조적 조회, 의존성 검사, 그래프 편집을 통한 정제까지 지원한다. MemP 역시 절차 기억 수명주기 연산이라는 강조점을 공유하지만 명시적인 절차 그래프 없이 궤적과 스크립트형 추상을 저장하고 조회한다.

2.4 절차 기억으로 읽는 절차 그래프: CoALA 관점

이 세 계열을 한 장으로 요약하는 착안이 CoALA(Cognitive Architectures for Language Agents)다. CoALA는 ACT-R과 SOAR의 고전적 기억 분류를 LLM 에이전트에 이식해 작업 기억은 현재 문맥을, 에피소드 기억은 과거 경험을, 의미 기억은 사실 지식을 담게 하고, 절차 기억은 어떻게 행동할지를 규정하는 기술과 절차를 담는다. GraphRAG 계열의 검색 증강은 의미 기억 위에서, Reflexion이나 ExpeL 같은 궤적 성찰은 에피소드 기억 위에서 작동한다.

네 분할 가운데 절차 기억만 가장 덜 명시적인 처우를 받아 왔다. 모델 가중치 안에 암묵적으로 남거나 프롬프트 템플릿, 스킴 라이브러리, 워크플로 스크립트 같은 임시 조각에 흩어져 있다. 절차 그래프는 상태 조건부 행동 전이를 조회와 갱신이 가능한 구조에 저장함으로써 CoALA가 가리킨 절차 기억 모듈을 실제로 구현한 것으로 읽을 수 있다.

2.5 여덟 차원으로 놓고 비교하기

원문 부록 A.5는 대표 방법 24개를 여덟 개 설계 차원으로 비교한다. 여덟 차원은 형태와 저장 단위, 출처, 갱신 가능 여부, 조회 방식, 에지 의미, 적용 범위, 그래프 여부다. 이 가운데 판별력이 큰 형태, 저장 단위, 출처, 갱신, 조회 방식, 그래프 여섯 개를 골랐고, 행도 본문에서 이름이 오간 대표 기법 13개로 추렸다.

표 2-1 대표 기법과 절차 그래프의 설계 차원 비교 (원문 부록 A.5의 8차원 가운데 6개로 축약)

기법	형태	저장 단위	출처	갱신	조회 방식	그래프
ReAct	없음	행동	없음	아니요	없음	아니요
Reflexion	텍스트	자기비판	궤적	예	프롬프트	아니요
MemoryBank	텍스트	요약	궤적	예	임베딩	아니요
RAP	텍스트	궤적	궤적	예	임베딩	아니요
ExpeL	텍스트	인사이트	궤적	예	임베딩	아니요
AutoGuide	텍스트	규칙	궤적	예	LLM 선택	아니요
AWM	텍스트+코드	워크플로	궤적	예	프롬프트	아니요
MemP	텍스트+궤적	절차	궤적	예	임베딩	아니요
KnowAgent	텍스트	행동	혼합	아니요	고정 프롬프트	일부
FlowBench	플로차트	워크플로	수작업	아니요	프롬프트	예
AFlow	코드	연산자	궤적	예	실행	예
ToolNet	그래프	도구	궤적	예	구조적	예
GraphRAG-Tool Fusion	그래프	도구	혼합	아니요	하이브리드	예
절차 그래프	그래프	행동 전이	혼합	예	하이브리드	예

표가 드러내는 차이는 무엇을 저장하는가, 어떻게 접근하는가, 무엇을 고칠 수 있는가의 세 질문으로 모인다. KnowAgent가 행동 전이 지식을 프롬프트 속 텍스트로 주는 동안 FlowBench는 분기 조건이 달린 플로차트를 포함한다. 개선 대상도 제각각이다. ToolNet은 궤적에서 도구 전이를 끌어내고 SkillGraph는 기술 은행 갱신을 에이전트 간 통신 그래프와 엮는 반면, 절차 그래프는 노드와 에지 편집으로 절차 그래프 자체를 갱신한다. 절차 그래프의 기여는 그래프 구조나 조건부 지식 그 자체가 아니라 속성이 붙은 전이, 국소화된 가이드нс, 구조적 정제의 조합에 있다.

2.6 절차 그래프의 차이

세 계열은 각자 비워 둔 지점이 하나씩 있다. 행동 선택 계열은 허용 전이를 암묵에 남기고, 구조화된 사전 계열은 조건 지식을 그래프에 온전히 엮지 못하며, 자기 개선 계열은 재사용 지식을 편집 가능한 절차 구조로 묶지 않는다. 절차 그래프는 이 세 지점을 동시에 메운다. 전이에 `condition guidance pitfalls` 속성을 붙여 허용 전이를 명시하고, 실행 중에는 현재 문맥에서 현재 절차의 위치를 좁혀 찾아 이웃한 전이만 조회하며, 실행 후에는 LLM 정제기가 토폴로지와 속성을 편집해 되돌려 준다. 어느 하나만으로는 기존 방법과 겹치지만 세 요소의 조합이 절차 그래프를 구분 가능한 설계로 만든다.

참고: 원문 §2 (p.3)·부록 A (pp.16~18)

03

3. 절차 그래프 프레임워크



이 장은 책 전체의 뼈대를 세운다. 절차 그래프(Procedural Graph, PG)는 절차 지식을 방향성 그래프에 담아 두 가지 일을 동시에 한다. 과업을 푸는 동안에는 에이전트의 실행을 온라인에서 이끌고, 과업 실행이 끝난 뒤에는 그래프의 토폴로지와 속성을 반복적으로 다룬다. 하나의 자료 구조가 실행 안내와 구조 개선을 함께 맡는 셈이다.

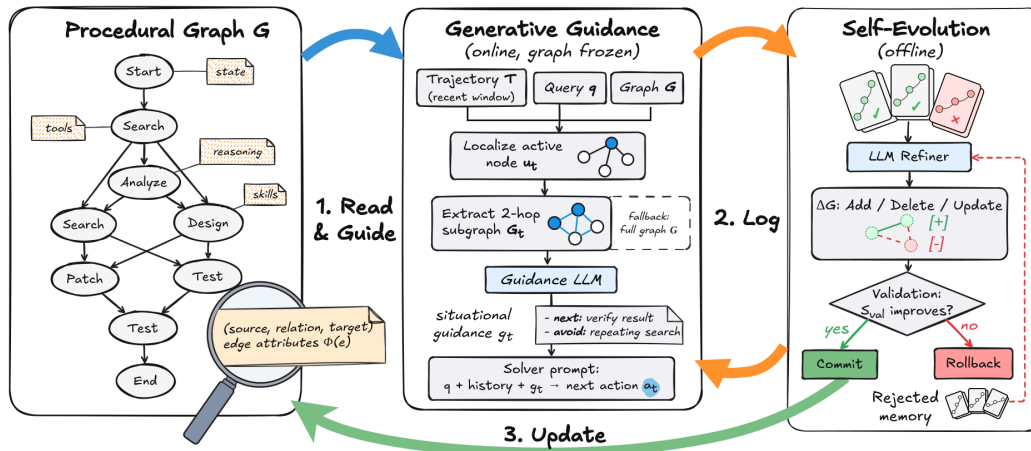


그림 3-1 절차 그래프 프레임워크 전체 — 원문 그림 2를 재수록했다

두 국면: 온라인 추론과 오프라인 진화

프레임워크는 서로를 보완하는 두 국면으로 움직인다. 온라인 추론 국면에서 그래프는 동결된다. 과업을 푸는 동안 누구도 그래프를 고치지 않으며, 에이전트는 절차 그래프와 자신의 궤적을 결합해 그 시점 상황에 맞는 동적 가이드를 만들어 낸다. 그림 3-1의 가운데 패널이 이 흐름을 담고 있다. 질의가 들어오면 현재 행동에 해당하는 활성 노드를 찾고, 그 이웃을 꺼내 가이드 모델에 넘기는 순서다. 그림 3-2는 같은 흐름을 한국어 라벨로 펼쳐 놓은 것이다.

매 단계마다 그래프 이웃이 상황 가이드언스로 바뀐다

온라인 추론 — 그래프는 동결된 채 레직과 결합한다

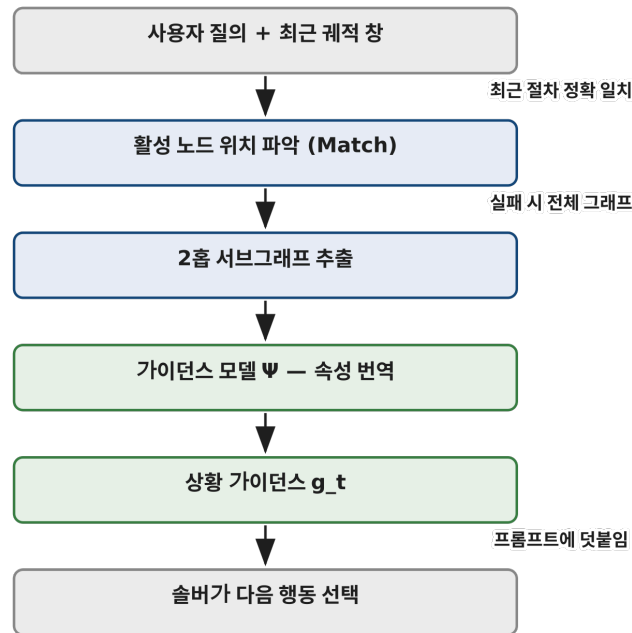


그림 3-2 생성형 가이드언스 — 활성 노드를 찾고 2홉 이웃을 펼쳐 가이드언스 모델이 단계별 상황 가이드언스로 번역해 솔버에 덧붙인다

오프라인 진화 국면은 학습 과업 배치의 실행이 끝난 뒤에 시작된다. LLM 정제기가 이번 배치에서 남긴 진단 흔적을 분석하고, 자동화된 피드백 루프를 통해 그래프의 토폴로지와 속성을 수정한다. 그림 3-1의 오른쪽 패널이 이 루프를 담당한다. 온라인 국면이 그래프를 소비하는 쪽이라면 오프라인 국면은 그래프를 생산하는 쪽이고, 두 국면이 번갈아 돌면서 그래프는 점점 단단해진다.

3.1 형식 표현

절차 그래프를 수식으로 적으면 방향성 속성 그래프 하나가 된다.

$$G = (V, R, E, \Phi), \quad E \subseteq V \times R \times V$$

여기서 V 는 추상 노드의 집합이고, R 은 전이 관계의 어휘이며, E 의 각 원소는 방향성과 속성을 갖춘 삼중조다. 에지 (u, r, v) 는 노드 v 가 관계 r 아래에서 노드 u 다음에 허용된다는 뜻이다. “ u 를 마쳤다면 v 로 갈 수 있다”는 문장을 그래프가 직접 기억하는 것이다.

노드가 추상화하는 대상은 넓다. 도구 함수 한 개가 노드가 될 수 있고, 스킬이나 내부 추론 단계, 과업의 상태도 노드가 된다. 속성 사상 Φ 는 에지마다 이름 붙은 속성 집합을 매핑하며, 스키마는 과업에 맞게 지정한다. 이 책이 다루는 구현은 세 개의 텍스트 필드를 쓴다. `condition`(적용 조건)은 그 전이가 언제 적용되는지 말하고, `guidance`(수행 지침)는 어떻게 나아가는지 알려 주며, `pitfalls`(주의점)은 무엇을 피해야 하는지 짚어 준다.

금융 플래닝 예시가 이해를 돕는다. 에지 `(cash_flow_forecast, LEADS_T0, fund_raising_request)`는 현금흐름 예측이 자금 조달 요청으로 이어지는 전이다. 이 에지에 속성을 붙이면 다음과 같아진다. `condition`에는 “예상 권웨이가 안전 버퍼 아래로 떨어질 때”가 들어가고, `guidance`에는 “자금 조달 지연을 고려해 요청을 일찍 제출한다”가, `pitfalls`에는 “요청이 진행 중일 때 두 번째 요청을 쌓지 않는다”가 담긴다. 검색 한 번으로 전이의 조건과 요령, 함정이 함께 나오는 셈이다.

과업 지식을 삼중조로 구조화하면 허용되는 전이가 명시적으로 드러나고, 에이전트는 유효한 도구 호출과 행동 수열 쪽으로 이끌린다. 그래프의 출발점은 두 가지다. 전문가 사전에서 초기화하거나 백지에서 시작할 수 있다. 두 구축 전략의 차이는 원문 §5.3이 실험으로 비교한다.

3.2 추론 시점의 생성형 가이드런스

전이 속성을 독립적으로 조회하는 방식은 겉보기에 자연스럽지만, 상위-k 유사도 검색 같은 조회는 절차 단계 사이의 연결을 놓치기 쉽다. submit에 대한 가이드언스를 검색하면서 앞선 check_answer 전이를 빼놓으면 어떻게 될까. 제출이라는 행동만 던져지고, 그 제출을 적절하게 만들어 주는 검증 단계는 사라진다. 연결된 이웃을 함께 조회하면 행동과 그 절차적 전제가 한 묶음으로 나온다.

생성형 절차 그래프 가이드언스는 이 문제를 세 연산으로 푼다. 위치를 좁혀 찾는 locate, 이웃을 꺼내는 extract, 가이드언스를 만드는 generate가 그것이다. 사용자 질의를 q , 의사결정 시점 t 까지 행동과 관측이 번갈아 쌓인 이력을 $T_t = (a_1, o_1, \dots, a_{t-1}, o_{t-1})$ 라 하자. 시작점 표식으로 $a_0 = \text{Start}$ 를 두므로 첫 단계는 $u_1 = \text{Start}$ 에 위치한다. 각 시점에서 다음을 계산한다.

$$u_t = \text{Match}(a_{t-1}, V), \quad G_t = \begin{cases} N_h(u_t), & u_t \neq \emptyset \\ G, & \text{otherwise} \end{cases}, \quad g_t = \Psi(G_t, q, T_{t-w:t})$$

Match는 에이전트의 최근 절차, 예컨대 마지막 도구 호출을 노드 집합 V 와 정확 일치시켜 현재 위치를 찾는다. 방향성 에지 이웃 $N_h(u_t)$ 는 u_t 자신과, 최대 h 홉까지 확장하며 도달하는 나가는 전이들을 담는다. $T_{t-w:t}$ 는 최근 w 스텝 창이고, Ψ 는 가이드언스 언어모델이다. 일치에 실패하면 전체 그래프 G 로 물러난다.

연결된 이웃을 건네주는 까닭은 단순하다. Ψ 가 전이를 위상적 문맥 속에서 읽을 수 있어야 하고, h 개 전이 앞까지 가능한 다음 수를 미리 따져 봐야 하기 때문이다. Ψ 는 주변 에지들의 정적 속성 $\Phi(e)$ 를 상황 가이드언스 g_t 로 번역하면서 에이전트의 즉각적 목표가 무엇인지, 형식상·논리상 어떤 오류를 피해야 하는지 짚어 준다.

이렇게 만든 가이드언스 g_t 는 솔버 프롬프트 뒤에 덧붙는다. 솔버는 질의와 궤적, 가이드언스를 놓고 다음 행동을 고른다.

$$a_t \sim P_{\text{solver}}(q, T_t, g_t)$$

여기서 중요한 점은 통합이 소프트라는 것이다. 가이드언스는 솔버의 선택을 강제하지 않고 편향시킬 뿐이므로, 에이전트는 그래프 G 에 새겨진 절차 구조 쪽으로 기울면서도 추론의 자유를 그대로 유지한다. 일치에 성공하면 지역 이웃을, 실패하면 전체 그래프를 쓰는 이 설계의 성능과 효율은 원문 §5.5가 평가하고, 실제 실행 사례는 원문 부록 F에 정리되어 있다.

3.3 절차 그래프의 자기진화

오프라인 자기진화 루프는 실행 피드백에서 그래프의 토폴로지와 속성을 고쳐 주므로, 수작업 설계의 부담을 덜어 준다. 초기 그래프를 g_0 , k 라운드를 거친 뒤 남은 그래프를 g_k 라 하자. 각 라운드는 직전 그래프 g_{k-1} 에서 출발하며, 게이트에서 거부된 후보는 다음 라운드의 시작 그래프가 될 수 없다는 규칙이 깔려 있다. $k = 1$ 부터 K 까지 진화 엔진은 네 단계 루프를 돌린다. 그림 3-3은 이 루프 전체를 한 장의 도면에 요약한다.

검증 게이트와 거부 메모리가 진화를 가둔다

오프라인 자기진화 — 거부된 후보는 다음 라운드의 출발점이 되지 않는다

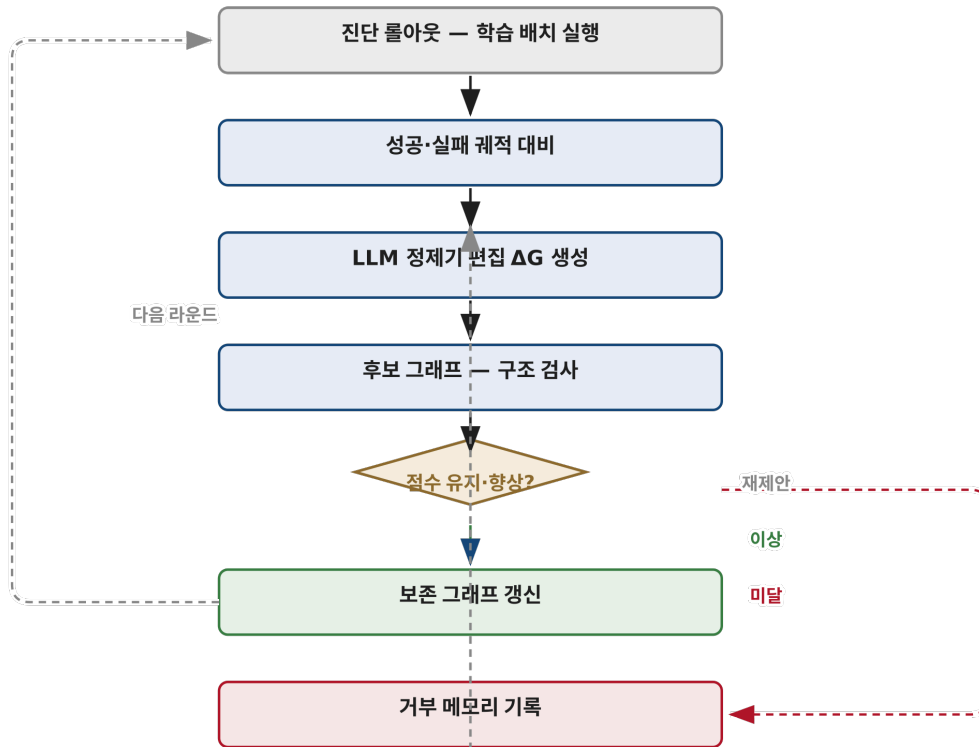


그림 3-3 자기진화 루프 — 진단 롤아웃의 성공·실패 궤적 대비에서 편집을 제안하고, 검증 게이트를 통과한 후보만 보존 그래프가 된다

1단계: 진단 롤아웃

보존 그래프 G_{k-1} 을 엮은 솔버가 학습 과업 배치 $B_k \subset D_{train}$ 을 실행한다. 실행마다 진단 흔적과 평가 점수를 함께 기록해 $E_k = \{(q_i, T_i^{(k)}, S_i^{(k)})\}$ 집합을 만든다. 여기서 $S_i^{(k)} \in [0, 1]$ 은 그 과업의 최종 점수다. 정제기는 점수 높은 흔적과 낮은 흔적을 나란히 놓고 비교하는데, 결과가 이진인 과업이라면 비교는 성공 대 실패로 단순해진다.

2단계: 피드백 기반 변이

오프라인 LLM 정제기가 나뉜 흔적들을 살펴 두 가지 패턴을 찾는다. 실패 궤적에서는 반복되는 오류 루프를, 성공 궤적에서는 여러 단계를 한 번에 건너뛰는 추론 지름길을 찾는다는 것이다. 이 피드백을 바탕으로 정제기는 구조화된 편집 집합 ΔG_k 를 만들어 내며, 여기에는 두 가지 토폴로지 편집이 들어간다. Add는 빠져 있던 검증 노드나 에지를 삽입해 구멍을 메우고, Delete는 궤적을 반복해서 실패로 물거나 진행 자체를 막는 노드와 에지를 제거한다.

속성 수정도 같은 편집 인터페이스를 통과한다. 정제기는 속성을 제자리에서 고치는 대신 해당 에지를 삭제한 뒤 갱신된 속성 값으로 다시 추가하며, 이 방식은 스키마가 정의한 어떤 속성에도 적용된다. 제안된 편집을 모두 적용하면 후보 그래프 $G_{cand_k} = G_{k-1} \oplus \Delta G_k$ 가 나온다. 연산 $\oplus$ 는 그래프 복사본에 편집을 적용하고 설정되어 있는 사이클 수리를 수행하므로, 원본이 변하지 않고 순환 구조도 정리된다.

3단계: 검증 게이트

구조를 고치는 변이가 학습 배치 너머의 성능을 실제로 끌어올리는지 판정하는 단계다. 편집 적용과 구조 검사를 통과한 후보만이 독립적인 홀드아웃 검증 집합 D_{val} 에서 평가를 받는다. 무효한 후보는 검증 롤아웃 이전에 폐기되므로 보존 그래프와 캐시된 검증 점수는 그대로 남는다. 검증 과업 점수의 평균은 다음처럼 계산한다.

$$S_{\text{val}}(G) = \frac{1}{|D_{\text{val}}|} \sum_{(q,y) \in D_{\text{val}}} S(f_{\text{solver}}(q | G), y)$$

초기 그래프는 기준 점수를 세우기 위해 한 번 평가된다. 구조적으로 유효한 후보가 도착하면 보존 그래프는 아래 규칙으로 갱신된다.

$$G_k = \begin{cases} G_k^{\text{cand}}, & S_{\text{val}}(G_k^{\text{cand}}) \geq S_{\text{val}}(G_{k-1}) \\ G_{k-1}, & \text{otherwise} \end{cases}$$

게이트는 측정된 검증 점수가 현 그래프의 캐시 점수 이상인 후보를 채택한다. 점수가 밀리는 후보는 받지 않는 보수적 규칙이며, 이 판정이 확률적 평가 환경에서 어떻게 작동하는지는 원문 §5.4가 자세히 살펴본다.

4단계: 거부 메모리

반복적인 자기수정에는 한 가지 함정이 있다. 정제기가 같은 실패 편집을 서로 다른 옷을 입혀 거듭 제안할 수 있다는 것이다. 검증 게이트에서 후보 그래프가 밀려나면 ($S_{\text{val}}(G^{\text{cand}}_k) < S_{\text{val}}(G_{k-1})$), 프레임워크는 후보 그래프와 제안된 편집, 그리고 이에 딸린 학습 궤적 E_k 와 검증 결과를 통째로 거부 메모리 H_{rejected} 에 기록한다.

정제기에 넘겨줄 궤적 문맥은 학습 궤적들을 이어 붙여 만든다. 설정된 최대 토큰 길이 L_{max} 를 넘으면 앞부분 토큰을 버리고 마지막 L_{max} 토큰을 원래 순서 그대로 보존한다. 궤적의 시작보다 끝을 남기는 선택이고, 이렇게 만들어진 문맥을 C_k 라 부른다. $k+1$ 라운드의 편집을 제안할 때 정제기는 거부 이력을 음성 증거로 함께 받는다.

$$\Delta G_{k+1} \sim P_{\text{refiner}}(G_k, C_{k+1}, H_{\text{rejected}})$$

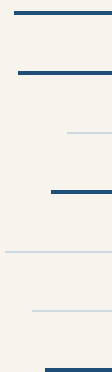
거부 이력 H_{rejected} 가 있으면 정제기는 이미 실패한 편집을 피해 다닌다. 물론 제안된 변경이 보존 그래프에 들어가는 길은 언제나 검증 게이트를 거친 뒤에 열린다. 진화가 아무리 대담해도 최종 문은 게이트가 지킨다.

참고: 원문 §3 (pp.3-6)

DAY

04

4. 실험 설계



공정한 비교는 조건을 고정하는 데서 시작한다. 저자들은 일곱 벤치마크, 일곱 베이스라인, 네 개 LLM으로 평가 환경을 꾸렸고, 모든 방법이 같은 솔버와 같은 디코딩 설정, 같은 학습 데이터를 쓰도록 맞췄다. 방법 사이에서 달라지는 유일한 축은 절차 경험을 어떻게 저장하고 다시 꺼내 쓰는가다. 구현 세부와 데이터 통계는 부록 B에 담겨 있다.

벤치마크 일곱 가지

일곱 벤치마크는 에이전트가 실무에서 부딪히는 과업 유형을 고르게 담는다.

- HotpotQA: 검색 도구를 써서 여러 문서를 넘나들며 답을 좁혀 가는 다중 홉 질의응답이다.
- MultiChallenge: 여러 턴에 걸친 대화에서 앞서 내려받은 지시를 끝까지 유지하는 능력을 본다.
- GDPval: 산출물을 전문가 리뷰어로 채점하는 개방형 전문 과업이다.
- ALFWorld: 엄격한 행동 순서를 지켜야 하는 가사 임무다.
- τ -bench: 실시간 사용자와 상호작용하면서 정책을 준수하는 도구 사용을 평가한다.
- BFCL v3: 다중 턴에 걸친 함수 호출을 다룬다.
- EnterpriseArena: 지연된 피드백과 거시 경제 충격 아래 이어지는 장기 금융 의사결정 시뮬레이션이다.

베이스라인 일곱 가지

모든 방법이 동일한 ReAct 솔버를 공유하며, 절차 경험의 저장과 재사용 방식만 다르다. 학습 기반 베이스라인은 자기진화 루프와 정확히 같은 학습 궤적을 소비한다. 비교의 공정성이 여기서 확보된다. 경험 구조가 단순한 것부터 복잡한 것까지 순서대로 나열하면 다음과 같다.

- Vanilla ReAct: 메모리가 없는 솔버다.
- MemoryBank: 망각을 갖춘 요약형 경험 저장소를 유지한다.
- RAP: 과거 궤적을 문맥 예시로 검색해 제공한다.
- ExpeL: 궤적을 자연어 인사이트로 증류한다.
- AutoGuide: 상태 조건 가이드라인을 상태에 맞춰 끌어온다.
- AWM: 성공 궤적에서 선형 워크플로를 유도한다.
- KnowAgent: 텍스트로 된 행동 전이 규칙을 유지한다.

모델과 디코딩

평가 대상 LLM은 Claude Sonnet 4.6, Gemini 3.1 Pro, Gemini 3.5 Flash, Grok 4.1 Fast 네 종류이다. 가이드언스 모델과 LLM 정제기는 언제나 솔버와 같은 LLM을 쓴다. 재현성을 위해 모든 호출은 temperature 0의 그리디 디코딩으로 통일했다.

절차 그래프 설정

온라인 가이드언스는 위치를 좁혀 찾은 활성 노드의 $h=2$ 홉 이웃과 최근 궤적 창 $w=3$ 을 사용한다. 구축 방식에 따른 비교는 원문 §5.3에서 다루고, 벤치마크별 그래프 통계는 아래에서 요약한다.

부록 B: 분할, 지표, 그래프 규모

표 4-1은 각 벤치마크의 도메인과 분할 규모를 보여준다.

표 4-1 데이터셋 분할과 규모 (원문 Table 5 재구성)

벤치마크	도메인	학습	테스트
HotpotQA	개방형 질의응답	1,000	1,000

벤치마크	도메인	학습	테스트
MultiChallenge	다중 턴 대화	100	166
GDPval	전문 산출물	88	44
ALFWorld	가사 임무	238	134
τ -bench	고객 응대	500	115
BFCL v3	함수 호출	100	100
EnterpriseArena	금융 시뮬레이션	50	50

자기진화 동안 학습 분할은 HotpotQA에서 $S=100$, MultiChallenge에서 $S=20$ 간격으로 순회한다. 검증 게이트가 쓰는 검증 집합은 학습·테스트 양쪽과 겹치지 않는 별도 홀드아웃이며, HotpotQA 1,000건, MultiChallenge 100건, EnterpriseArena 20에피소드다. EnterpriseArena의 각 설정은 학습 50, 테스트 50 에피소드를 돌리고, 두 쪽이 서로 다른 난수 시드 집합을 쓴다.

지표도 과업마다 다르다. HotpotQA는 Gemini 3.1 Pro 판정 모델이 질문·정답·에이전트 답을 비교해 동치 여부를 가리는 LLM 채점 정확도를 쓴다. MultiChallenge는 LLM 판정으로 전체 성공률과 함께 추론 메모리, 지시 유지, 버전 편집 신뢰성, 자기 일관성 네 축을 점수 매긴다. GDPval은 과업별 루브릭 평균 점수, ALFWorld는 테스트 게임 성공률이다. τ -bench는 Pass@1, 곧 최종 데이터베이스 상태가 목표 상태와 일치한 에피소드 비율을, BFCL v3는 공식 다중 턴 정확도를 보고한다. EnterpriseArena는 전체 기간 생존율, 평균 수명, 시간 평균 기업 점수, 평균 조달 자금을 측정한다.

그래프 규모는 의외로 작다. 함수 카탈로그가 커서 131개 노드를 가진 BFCL v3를 빼면 모든 그래프가 노드 7~17개, 삼중 조 7~27개 안에 들어온다. 관계 어휘는 LEADS_TO, TRIGGERS, PROVIDES_INPUT_FOR, CONVERGES_TO 네 유형뿐이다. 대부분의 에지가 condition, guidance, pitfalls 세 속성을 온전히 갖추며, 그 가운데서도 guidance 필드가 가장 충실하게 채워진다. 절차 지식을 잡아내는 데 거대한 그래프가 필요 없다는 사실이 이 숫자 자체로 말해준다.

참고: 원문 §4 (p.6)·부록 B (pp.19–21)

DAY

05

5. 주요 결과



여섯 벤치마크와 네 개 LLM이 만나는 24개 모델-벤치마크 설정 가운데 절차 그래프 기법은 21개에서 1위 또는 공동 1위를 기록했다. 각 설정에서 가장 강한 베이스라인과 하나씩 겨뤘 보면 성적표는 19승 2무 3패다. 무승부를 제외한 단측 정확 부호검정은 다음 값을 준다.

$$p = 4.3 \times 10^{-4}$$

이 값은 승패 배열을 우연타로 치부하기 어렵다는 뜻이다. 격차가 가장 큰 세 곳은 BFCL v3와 Gemini 3.5 Flash 조합(67.00% 대 58.00%, +9.00포인트), GDPval과 Gemini 3.1 Pro 조합(78.78 대 71.37, +7.41포인트), 같은 모델이 나선 τ -bench(80.00% 대 73.04%, +6.96포인트)다.

표 5-1은 원문 Table 1을 축약해 설정별 최강 베이스라인과의 격차만 보여준다. 원문이 함께 실은 95% 신뢰구간은 생략했다.

표 5-1 벤치마크별 설정 최강 베이스라인 대비 절차 그래프 성적 (원문 Table 1 축약 재구성)

벤치마크	모델	최강 베이스라인	절차 그래프	차이(포인트)
HotpotQA	Claude Sonnet 4.6	AutoGuide 75.40	74.50	-0.90
HotpotQA	Gemini 3.1 Pro	ExpeL 86.00	87.30	+1.30
HotpotQA	Gemini 3.5 Flash	ExpeL 83.40	84.50	+1.10
HotpotQA	Grok 4.1 Fast	AWM 74.30	74.40	+0.10
MultiChallenge	Claude Sonnet 4.6	AWM 89.76	89.76	0.00(동점)
MultiChallenge	Gemini 3.1 Pro	MemoryBank 95.18	95.78	+0.60
MultiChallenge	Gemini 3.5 Flash	AWM 91.57	91.57	0.00(동점)
MultiChallenge	Grok 4.1 Fast	MemoryBank 84.94	86.75	+1.81
GDPval	Claude Sonnet 4.6	MemoryBank 50.13	51.49	+1.36
GDPval	Gemini 3.1 Pro	RAP 71.37	78.78	+7.41
GDPval	Gemini 3.5 Flash	ExpeL 62.45	64.42	+1.97
GDPval	Grok 4.1 Fast	KnowAgent 68.15	71.19	+3.04
ALFWorld	Claude Sonnet 4.6	ExpeL 91.79	93.28	+1.49
ALFWorld	Gemini 3.1 Pro	AWM 99.25	100.00	+0.75
ALFWorld	Gemini 3.5 Flash	ExpeL 90.30	94.03	+3.73
ALFWorld	Grok 4.1 Fast	ExpeL 42.54	39.55	-2.99
τ -bench	Claude Sonnet 4.6	ExpeL 71.30	73.91	+2.61
τ -bench	Gemini 3.1 Pro	RAP 73.04	80.00	+6.96
τ -bench	Gemini 3.5 Flash	KnowAgent 38.26	44.35	+6.09
τ -bench	Grok 4.1 Fast	KnowAgent 68.70	67.83	-0.87
BFCL v3	Claude Sonnet 4.6	RAP 65.00	67.00	+2.00
BFCL v3	Gemini 3.1 Pro	AWM 64.00	66.00	+2.00
BFCL v3	Gemini 3.5 Flash	ExpeL 58.00	67.00	+9.00
BFCL v3	Grok 4.1 Fast	MemoryBank 56.00	60.00	+4.00

표에서 읽히는 흐름은 세 갈래다. 첫 갈래는 보편성이다. GDPval과 BFCL v3에서 PG는 네 개 LLM 전부에서 일곱 베이스라인을 모두 앞선다. 이 두 과업에서의 우위는 특정 솔버에 묶여 있지 않다. 두 번째 갈래는 다중 턴 대화의 안정성이다. MultiChallenge에서 네 모델 모두 1위 또는 공동 1위를 차지했는데, Claude Sonnet 4.6에서는 AWM과, Gemini 3.5 Flash에서는 AWM과 KnowAgent와 어깨를 나란히 한다. 세 번째 갈래는 베이스라인 쪽이다. 순위가 과업과 모델에 따라 뒤엎히고, 늘 2위를 지키는 방법은 없다. 어떤 단일 기법도 만능 해법이 아님을 이 뒤엎킴이 다시 확인해 준다.

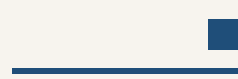
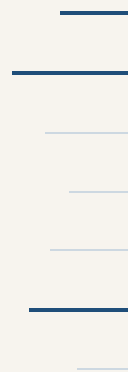
HotpotQA는 다른 그림이다. 최강 베이스라인 대비 격차가 -0.90 포인트에서 $+1.30$ 포인트 사이에 머문다. ALFWorld의 Grok 4.1 Fast와 τ -bench의 같은 모델에서도 근소하게 밀렸다. 저자들 스스로 이득의 크기가 벤치마크에 따라 크게 갈린다고 인정한다. 조건부 가이드스, 재사용 가능한 행동 시퀀스, 명시적 전이를 하나의 연결 그래프로 묶는 구성은 다양한 환경에서 도움이 되지만, 절차 구조가 성패를 실제로 가르는 과업에서 힘이 집중된다. 검색형 질의응답처럼 절차보다 사실 회수가 지배하는 과업에서는 개선 폭이 작아진다. 결과를 읽는 독자도 같은 잣대를 들고 와야 한다.

참고: 원문 §5.1 (pp.7-8)

DAY

06

6. 장기 의사결정과 생존



132개월짜리 파산 게임

에이전트의 진짜 시험대는 한 번의 벤치마크 점수가 아니라 시간이다. 원문은 이 지점을 재기 위해 EnterpriseArena라는 시뮬레이터를 배치한다. 에이전트는 마이크로파이낸스 대출 기관의 최고재무책임자 역할을 맡아 최대 132개월에 걸쳐 월 단위 금융 의사결정을 내린다. 엄격한 유동성 제약이 걸려 있고, 에이전트에 공개되지 않은 세 차례의 거시 위기가 일정에 잡혀 있다.

시뮬레이터의 상태는 현금 잔액, 총 대출 포트폴리오, 대출손실 총당금, 이자·원금 수취액, 지급 계정, 총 부채, 조달된 자본, 활성 사용자 수를 담는 다차원 튜플로 정리된다. 에이전트가 쓸 수 있는 핵심 행동은 두 가지다. `book_closing()`은 한 달을 진행시키면서 대출 상각, 운영 비용과 사용자 획득 비용 차감, 부실 대출 상각과 새 총당금 산정을 실행한다. `fund_raising_request(type, amount)`는 자본을 요청하는데, 현실적인 제약 두 개가 붙는다. 자본은 즉시 도착하지 않고 요청 시점에서 1~6개월의 확률적 지연을 거쳐 들어오며, 한 번에 조달할 수 있는 규모도 거시 경제 상황과 기관 재무 건전성에 따라 동적으로 제한된다. 현금 잔액이 음수가 되는 순간 시뮬레이션은 파산으로 즉시 끝난다.

세 위기는 강도와 성격이 다르다. 첫 위기(32개월)는 상환율이 98%에서 90%로 내려가는 완만한 수축이다. 둘째 위기(59개월)는 심각한 경기 침체라서 상환율이 60%로 떨어지고, 상각이 급증하며, 사용자 자연 증가가 음수로 돌아선다. 셋째 위기(112개월)는 시스템 전체의 유동성 동결이다. 상환율이 40%까지 내려가고 조달 한도도 극단적으로 조여 새 자본 확보가 극도로 어려워진다.

측정 지표도 정해져 있다. 전체 기간 생존율(full-horizon survival)은 파산 없이 132개월을 완주한 실행의 비율이고, 위기 별 열은 32·59·112개월 도달 비율이다. 평균 개월은 조기 종료를 포함한 실행 길이 평균이며, 도구 호출/월은 메모리 연산과 상태 변경 행동을 뺀 정보 조회 호출을 개월 수로 나눈 값이다. 조달액은 실행별로 실제 수령한 자본의 누적 평균이다.

생존율과 수명이 함께 움직인다

원문 그림 3을 그림 6-1로 재수록한다. 네 LLM의 Kaplan–Meier 생존 곡선과 앙상블 현금 궤적이 나란히 놓인다. 절차 그래프는 네 모델 모두에서 최고이거나 공동 최고의 전체 기간 생존율과 최장 평균 수명을 기록한다. 생존율은 Claude Sonnet 4.6에서 44.0%에서 58.0%로, Gemini 3.1 Pro에서 6.0%에서 34.0%로, Grok 4.1 Fast에서 26.0%에서 40.0%로 올라간다. Gemini 3.1 Pro의 절대 상승폭 28포인트가 가장 크고, Claude Sonnet 4.6과 Grok 4.1 Fast는 각각 14포인트씩 오른다. Grok 4.1 Fast에서는 평균 기업 점수 \$39.62M로 전 구성 최고를 낸다.

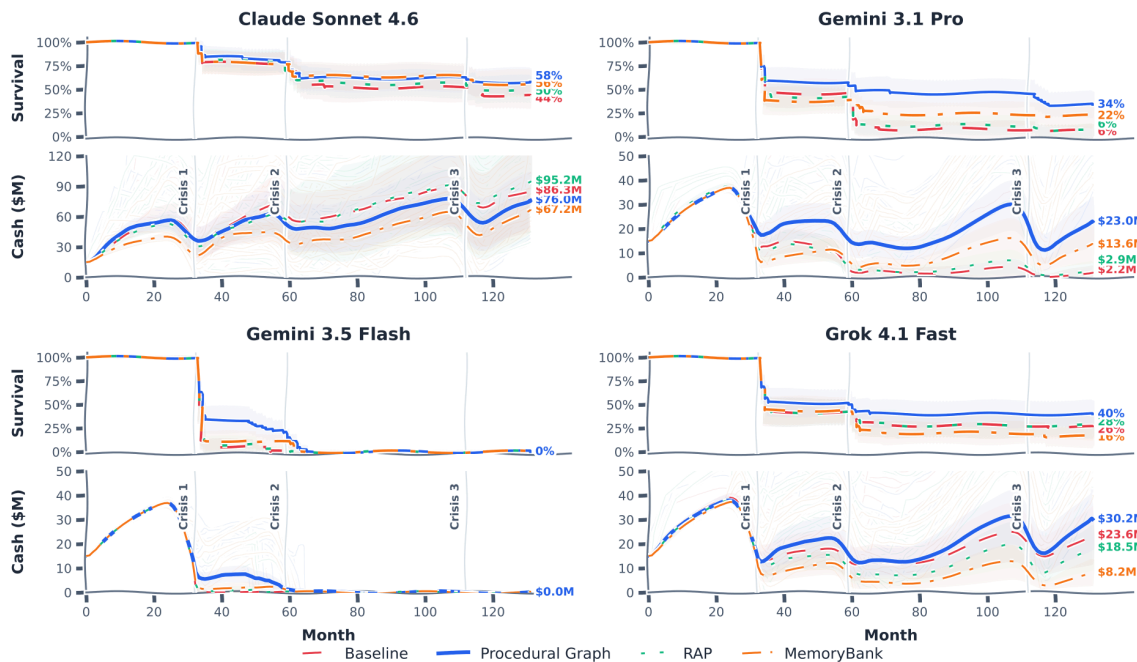


그림 6-1 네 LLM의 생존 곡선과 현금 궤적 — 원문 그림 3을 재수록했다

표 6-1 비유도 베이스라인과 절차 그래프 구성의 EnterpriseArena 핵심 지표(1차 위기 도달율은 네 모델 모두 100.0%라 열에서 뺐다). 조달액과 도구 호출은 실행당 월평균 값이다.

모델·설정	전체 기간 생존율	평균 개월	평균 점수	2차 위기	3차 위기	도구 호출/월	조달액
Claude Sonnet 4.6 베이스라인	44.0%	89.80	\$78.86M	78.0%	52.0%	0.13	\$152.12M
Claude Sonnet 4.6 + PG	58.0%	98.58	\$70.38M	80.0%	60.0%	0.36	\$130.44M
Gemini 3.1 Pro 베이스라인	6.0%	50.28	\$31.85M	38.0%	8.0%	0.89	\$21.82M
Gemini 3.1 Pro + PG	34.0%	79.22	\$37.21M	54.0%	46.0%	3.18	\$38.20M
Gemini 3.5 Flash 베이스라인	0.0%	33.58	\$28.59M	0.0%	0.0%	18.94	\$0.00M
Gemini 3.5 Flash + PG	0.0%	40.62	\$29.08M	14.0%	0.0%	12.53	\$9.39M
Grok 4.1 Fast 베이스라인	26.0%	63.76	\$39.42M	42.0%	28.0%	0.47	\$27.24M
Grok 4.1 Fast + PG	40.0%	75.14	\$39.62M	50.0%	40.0%	0.40	\$30.11M

가이드스가 바꾸는 것은 호출 개수가 아니라 타이밍이다

가이드스가 바꾸는 것은 도구를 몇 번 부르는가가 아니라 무엇을 언제 부르는가다. 가이드스 없는 Gemini 3.5 Flash는 한 턴 안에서 현금과 시장 상태를 거듭 조회해 중복 관측을 문맥에 쌓는다. 월평균 18.94회의 도구 호출이 나오는데, 그래프는 이를 12.53회로 줄이면서 평균 기업 점수는 오히려 끌어올린다.

Claude Sonnet 4.6과 Gemini 3.1 Pro에서는 방향이 반대다. 도구 호출이 월평균 0.13에서 0.36으로, 0.89에서 3.18로 늘어나는데 생존율도 함께 오른다. 그래프가 자금 결정 전에 예측 계산과 시장 점검을 돌리도록 이끌기 때문이다(원문 부록 C.2).

생존과 상관되는 행동: 선제적 자금조달

네 모델 전체에서 생존과 함께 움직이는 행동은 선제적 자금조달(anticipatory fundraising)이다. 자본이 요청 후 1~6개월 뒤에야 도착하므로 위기를 넘기려면 유동성이 바닥나기 훨씬 전에 요청해 줘야 한다. 그림 6-2는 이 시간 차를 두 궤적으로 대비한다. 전체 궤적을 보면 가이드언스 없는 Gemini 3.5 Flash 베이스라인은 조달을 너무 늦게 시작한다. 평균 조달액이 \$0.00M에 그친다. PG 가이드언스를 붙인 Flash는 \$9.39M, PG 가이드언스 Grok 4.1 Fast는 \$30.11M을 조달한다.

위기를 넘기는 행동은 선제적 자금조달이다

자본이 도착하는 데 1~6개월이 걸리므로 요청은 위기보다 앞서야 한다



EnterpriseArena 월 (최대 132개월)

그림 6-2 선제적 자금조달 — 가이드언스 있는 에이전트는 안정기에 요청해 위기 전에 완충을 확보하고, 없는 에이전트는 조달을 늦춰 고갈한다

첫 위기 앞의 세 에이전트

숫자 뒤의 메커니즘을 보려면 같은 과제 인스턴스에서 세 에이전트가 첫 위기에 들어서는 궤적을 나란히 놓아봐야 한다. 원문 부록 C.3는 Grok 4.1 Fast가 동일 인스턴스(시드 14)에서 29~33개월 사이에 남긴 실제 실험 로그를 추려 보여준다.

베이스라인 에이전트는 근시와 규칙 위반을 함께 보인다. 31개월에 현금 \$14.2M을 확인하고도 파산 위험이 없다고 판단해 그냥 한 달을 넘긴다. 32개월에 현금이 \$6.2M로 급감하자 equity \$20M을 요청하고, 33개월에 현금이 \$683K로 떨어진 뒤에야 앞의 equity 요청이 아직 처리 대기 중인데 debt \$10M을 추가로 낸다. 환경은 조달 요청을 하나만 대기시킬 수 있다는 제약 때문에 이 요청을 거절한다. 조달을 위기 직전까지 미룬 대가가 그대로 드러난다.

메모리 요약 에이전트는 과거 사건을 LLM이 만든 요약으로 넘겨 본다. 현금 소진이 빨라지는 것은 알아채지만, 추론에서는 \$18M과 \$20M이라고 말하면서 실제 요청에는 단위 없는 18과 20을 넣고, 앞선 요청이 대기 중인 동안 같은 요청을 거듭 제출해 거절을 되풀이한다.

PG 가이드언스 에이전트는 다르게 움직인다. 27개월에 넣은 equity 요청이 시장 한도에 걸려 \$28.8M으로 줄어든 채 30개월에 도착한다. 이 에이전트는 같은 달에 VIX가 10.51로 매우 낮아 시장이 평온하고 희석 비용이 적다고 판단, \$50M을 추가 요청한다. 이어서 12개월 현금흐름 예측으로 runway을 확인하고, 대기 중인 자본이 도착하기를 기다리는 동안 `book_closing()`으로 한 달씩 넘긴다. 전송 지연과 단일 대기 제약을 끝까지 지키며 위기 구간을 건너 132개월 시뮬레이션 끝까지 살아남는다. 대기 중인 조달을 추적하고, 예상 runway을 점검하고, 도착을 기다리며 달을 진행시키는 일이 베이스라인과 메모리 요약 궤적의 무효 요청과 정확히 대비된다.

원문은 이 관찰에 선을 긋는다. 세 궤적은 같은 인스턴스에서 나타난 행동 차이를 보여줄 뿐, 각 행동이 실행 전반에서 얼마나 자주 나오는지는 정하지 않는다.

참고: 원문 §5.2 (p.8)·부록 C (pp.24~28)

DAY

07

7. 그래프 구축 전략



다섯 가지 구축 모드

그래프를 어디서 출발시키고 얼마나 갱신하느냐는 실무에서 가장 먼저 부딪히는 선택이다. 원문은 비유도 베이스라인을 기준으로 다섯 가지 구축 모드를 비교한다. 축은 두 개다. 전문가 사전(expert prior)에서 출발하느냐 백지(scratch)에서 출발하느냐, 그리고 고정이나 1회 갱신이나 반복 진화냐.

- 모드 1 수작업 전문가(Hand-crafted Expert): 사람이 도구 노드와 유효 전이, 자연어 가이드를 손으로 설계한 그래프를 테스트셋에 제로샷으로 돌린다. 갱신도 검증도 없다.
- 모드 2 전문가+1회 정적 갱신(Expert + Static Update): 모드 1의 그래프로 학습 분할 전체를 한 번에 훑아 궤적을 모으고, LLM 정제기가 큰 문맥 창 한 번으로 전이와 가이드를 일괄 갱신한다. 검증 안전장치는 없다.
- 모드 3 전문가+온라인 진화(Expert + Online Evolution): 모드 1에서 출발해 학습 분할을 순차 배치로 나누고 (HotpotQA는 100샘플, MultiChallenge는 20샘플), 배치마다 실패 로그로 노드·전이·지역 가이드를 갱신하며, 후보를 홀드아웃 검증으로 평가해 성능이 떨어지면 이전 최고 그래프를 복원한다.
- 모드 4 백지+정적 구축(Scratch + Static Build): 전문가 사전 없이 백지에서 정적으로 그래프를 만든다.
- 모드 5 백지+온라인 진화(Scratch + Online Evolution): 백지에서 출발해 모드 3과 같은 반복 진화 루프를 돌린다.

완전한 진화 루프, 즉 반복 갱신과 검증 게이트를 함께 갖춘 구성은 모드 3과 모드 5뿐이다. 모드 1은 고정, 모드 2와 모드 4는 1회성이다. 온라인 진화는 학습 배치 사이의 점진적 그래프 갱신을 뜻하며, 에피소드 안과 테스트 평가 중에는 그래프가 고정된다.

표 7-1 구축 모드별 성능 — 앞 두 열은 HotpotQA, 나머지는 MultiChallenge 지표다.

구축 모드	Ans EM	Ans F1	메모리 유 지	지시 편집	응답 신뢰 성	자기 일관성	전체 성
비유도 베이스라인(PG 없음)	58.8	71.21	86.96	86.67	71.43	100.00	87.50
모드 1 수작업 전문가	62.80	76.61	52.17	66.67	42.86	72.73	58.93
모드 2 전문가+1회 정적 갱신	63.80	77.16	47.83	53.33	28.57	81.82	53.57
모드 3 전문가+온라인 진화	63.10	76.34	95.65	100.00	85.71	81.82	92.86
모드 4 백지+정적 구축	55.40	69.49	91.30	86.67	85.71	90.91	89.29
모드 5 백지+온라인 진화	66.30	78.79	95.65	93.33	71.43	90.91	91.07

백지에서 출발한 진화가 전 구성을 앞선다

전문가 사전에서 출발한 모드 1~3과 백지에서 출발한 모드 4~5를 나눠 보면 그림이 읽힌다. HotpotQA에서 모드 5(백지+온라인 진화)가 전 구성 최고다. Ans F1 78.79%, Ans EM 66.30%로 비유도 베이스라인 대비 각각 +7.58 F1포인트, +7.50 EM포인트를 얻는다. 전문가 초기화 세 구성의 Ans EM도 62.80에서 63.80 사이로 베이스라인 58.8을 넘기므로, 전문가 사전이 아무런 가치가 없다는 뜻은 아니다. 대화 턴에 걸쳐 여러 제약을 유지해야 하는 MultiChallenge에서는 모드 5가 인간 사전 없이 전체 성공률 91.07%를 찍고, 모드 3(전문가+온라인 진화)이 92.86%로 전체 최고다. 인간의 출발 점이 있느냐 없느냐보다 반복 진화 루프를 갖췄느냐가 성능을 가른다.

결함 있는 전문가 사전을 되돌리는 루프

전문가 사전이 늘 옳은 것은 아니라는 사실이 이 표의 가장 큰 교훈이다. MultiChallenge에서 손으로 만든 전문가 그래프(모드 1)는 전체 성공률을 87.50%에서 58.93%로 떨어뜨린다. 1회 정적 갱신(모드 2)은 53.57%로 더 떨어뜨린다. 검증 게이트 없이 한 번에 갱신하면 전문가의 결함을 그대로 굳히거나 악화시킨다는 뜻이다. 이 상태에서 반복 진화와 검증 게이트를 결합한 모드 3이 92.86%로 회복한다. 전문가 초기화 대비 +33.93포인트다. 실패 궤적을 근거로 후보를 뽑되 홀드아웃 검증 성능이 지켜지는 편집만 채택하는 루프가, 처음에 성능을 깎아 내린 전문가 사전조차 되돌려 놓는다.

■ 자원과 안정성

부록 D.1의 자원·안정성 통계는 성능 뒤의 비용을 보여준다. 파싱 실패 측면에서 MultiChallenge의 비유도 베이스라인은 샘플당 1.05회를 기록하는데, 모드 5는 0.57회로 45.7% 줄인다. HotpotQA에서는 방향이 반대로, 전문가 초기화 모드 1~3이 백지 모드 4~5보다 실패가 적다(모드 2가 0.005로 최저, 모드 1이 0.007, 모드 3이 0.009, 모드 4·5는 각 0.016). 초기화 방식과 행동 포맷의 결합은 과제마다 다르며, 특정 그래프 스키마가 감소를 낳았다고 단정할 근거는 되지 않는다.

단계 수와 지연도 함께 읽어야 한다. MultiChallenge에서 모드 5는 샘플당 5.05단계, 92.81초인데 반해 모드 4는 8.02단계, 166.57초를 쓴다. 모드 2가 4.32단계, 65.93초로 가장 적지만 성공률도 최저 53.57%다. 최고 성공률의 모드 3은 6.73단계, 128.50초다. 손으로 만든 모드 1(6.60단계, 117.70초)과 비교하면 모드 5가 더 적은 단계와 시간으로 더 높은 성공률을 낸다. HotpotQA에서 모드 5의 31.53초는 PG 구성 가운데 최저 지연이다.

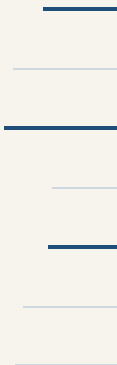
토큰 비용에서 전문가 작성 가이드선의 overhead가 드러난다. MultiChallenge에서 모드 1과 모드 3은 샘플당 약 11.0k, 12.2k 토큰을 쓰는 반면 모드 5는 7,984.50으로 비유도 베이스라인의 7,403.98에 근접한다. 모드 4는 14,859.80이다. HotpotQA에서는 이 비교가 뒤집혀 모드 5가 10,115.89, 모드 4가 6,396.90을 쓴다. 원문은 총 토큰이 단계 수와 생성물을 포함한 상호작용 전체를 반영하므로, 이 숫자만으로 그래프 표현이 더 압축적이라고 결론 내릴 수 없다고 못박는다. 자원 통계의 요점도 같은 맥락이다. 집계 수치만으로는 어떤 그래프 편집이 차이를 만들었는지 특정할 수 없으므로, 자원 소모는 언제나 과제 성능과 나란히 읽어야 한다.

참고: 원문 §5.3 (p.9)·부록 D (pp.29~30)

DAY

08

8. 자기진화 10라운드



EnterpriseArena는 에이전트를 재무책임자 자리에 앉힌다. 거시 경제 위기가 연이어 닥치는 가운데 기업 유동성을 관리해야 하고, 시뮬레이션이 끝날 때까지 회사를 살려 두는 것이 과업이다. 이 환경이 요구하는 핵심 판단은 조달 타이밍이다. 자금 조달을 요청하면 자본이 도착하기까지 한 달에서 여섯 달이 걸리므로, 현금이 바닥난 뒤에 움직이면 이미 늦었다.

가이드선 없는 베이스라인은 이 판단에 실패한다. 검증 분할에서 전체 기간 생존율은 0.0%, 평균 수명은 34.8개월에 그친다. 실패 지점을 들여다보면 사정이 더 분명해진다. 첫 번째 위기는 에이전트 100.0%가 통과하지만 두 번째와 세 번째 위기에서는 하나도 살아남지 못한다. 한 차례 위기는 견뎌도, 조달 시점을 경험에서 배워야 하는 연쇄 위기 앞에서 무구조 에이전트는 무너진다.

자기진화 루프는 이 출발선에서 열 라운드를 돈다. 원문 그림 4를 그림 8-1로 재수록한다. 라운드별 평균 수명과 조달 자본의 궤적이 그대로 담긴다. 검증 성적은 몇 차례의 뚜렷한 도약으로 이어지고, 그 사이에는 아무 것도 채택되지 않는 구간이 끼어 있다.

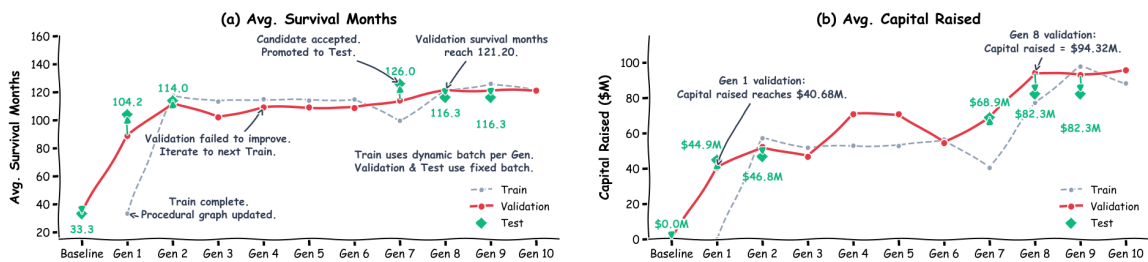


그림 8-1 자기진화 열 라운드 동안의 평균 수명과 조달 자본 — 원문 그림 4를 재수록했다

가장 큰 단일 도약은 라운드 1이 만든다. 변이 엔진이 발견한 것은 정교한 전략이 아니라 순차적 백본이다. 자금 결정에 앞서 현금을 점검하고 런웨이를 예측하라는 순서를 그래프에 박아 넣었을 뿐인데 검증 생존율이 0.0%에서 45.0%로 뛴다. 라운드 2는 이 그래프가 남긴 흔적을 재사용한다. 매월 저장한 노트를 다음 달 초에 다시 읽는 recall_notes가 추가되면서 생존율이 80.0%에 올라탄다. 도구 사용도 함께 줄여 비유도 베이스라인의 월평균 17.23회에서 3.08회로 떨어진다.

라운드 3부터 6까지는 채택이 없다. 세 라운드는 검증 성적 되돌림으로 기각되고, 한 후보는 롤아웃 전 구조 검증에서 잘못됐음이 드러나 시뮬레이션까지 가지 못한다. 라운드 7은 가지치기다. “아무것도 하지 않는” pass_action 분기를 잘라 낸다. 라운드 8은 행정 우회를 도입해 검증 생존율 90.0%에 도달하고, 라운드 9가 이 수준을 유지한다. 라운드 10 후보가 거부되면서 루프는 마지막으로 채택된 그래프를 남기고 종료한다. 그림 8-2는 이 열 라운드를 채택과 거부의 타임라인으로 정리한다.

열 라운드의 채택과 거부가 생존율을 세운다

채택은 드물게, 거부는 기록으로 — 개별 판정은 탐색 흔적이다



자기진화 라운드 (1-10)

그림 8-2 자기진화 10라운드 — 라운드 1의 순차 백본과 라운드 2의 노트 재사용이 큰 도약이었고, 라운드 8의 행정 우회까지 검증 생존율 90%에 도달했다

표 8-1은 열 라운드의 검증 분할 결과를 베이스라인과 함께 모아 보여준다(원문 Table 11 축약 재구성).

라운드	판정	생존율	평균 수명(개월)	월평균 도구 호출	조달 자본
베이스라인	출발점	0.0%	34.80	17.23	\$0.47M
라운드 1	채택	45.0%	88.90	7.92	\$40.68M
라운드 2	채택	80.0%	112.60	3.08	\$52.33M
라운드 3	기각	65.0%	102.15	3.30	\$46.74M
라운드 4	기각	75.0%	109.20	3.05	\$70.85M
라운드 5	롤아웃 전 실패	75.0%*	109.20	3.05	\$70.85M
라운드 6	기각	65.0%	108.75	2.91	\$54.54M
라운드 7	채택	80.0%	114.15	3.12	\$68.91M
라운드 8	채택	90.0%	121.20	3.13	\$94.32M
라운드 9	채택	90.0%	121.20	3.08	\$93.05M
라운드 10	기각, 루프 종료	85.0%	121.20	3.01	\$95.76M

* 라운드 5는 구조 검증 실패로 평가를 건너뛰어 라운드 4 수치를 그대로 이어받는다.

탐색이 돌려준 그래프와 탐색 도중의 중간 후보는 구분해서 보고해야 한다. 반환된 그래프의 테스트 생존율은 85.0%로 베이스라인의 0.0%를 압도한다. 피셔 정확 검정은 다음 값을 준다.

$$p = 2.6 \times 10^{-8}$$

탐색 중 관측된 최고 단일 라운드는 95.0%였다. 그러나 이 숫자를 대표값으로 내세우면 테스트 집합에서 골라 낸 것이 되므로 저자들은 85.0%를 보고한다. 읽는 방법에 대해서도 원문이 스스로 해명한다. 분할마다 에피소드가 20개뿐이라 개별 채택과 거부는 한두 에피소드 차이로 갈릴 수 있다. 따라서 이 표는 유의성 검정의 나열이 아니라 탐색의 흔적으로 읽어야 한다.

8.1 계산 효율

진화된 그래프는 점수 외에 중복된 환경 상호작용도 줄인다. 검증 분할의 월평균 도구 호출 수는 베이스라인 17.23회에서 라운드 2에 3.08회로 떨어지고 라운드 8까지 3.13회로 안정된다. 감소율은 81.8%다. 원인은 단순하다. 백지 ReAct 에이전트는 같은 턴 안에서도 현금 잔고와 시장 지표를 환경에 거듭 묻는다. 진화된 그래프는 같은 질의를 주기마다 한 번만 거치도록 순서를 강제한다. 저자들이 여기서 토큰이 아니라 도구 호출을 보고하는 이유도 밝혀져 있다. 시뮬레이션 로그는 호출 횟수는 기록하지만 월별 토큰 소모는 기록하지 않기 때문이다.

8.2 후보 선별 세 사례

진화 루프가 어떻게 실패한 후보를 걸러 내는지 세 사례가 보여준다.

- 성능 되돌림(라운드 3, 4, 6): 라운드 3에서 변이 엔진은 자금 조달을 촉발하는 현금 임계값을 낮췄다. 특정 학습 시드에서는 통했지만 검증 시드에서는 이른 지분 희석과 현금 부족을 낳아 검증 생존율이 15.0포인트, 평균 수명이 10.45개월 떨어졌다. 게이트는 후보를 기각하고 라운드 2 체크포인트를 복원했다.
- 실행 제약 되돌림(라운드 5): 후보가 시뮬레이션 전 구조 검증에 실패해 검증 평가를 건너뛰었다. 이 라운드의 검증 지표는 라운드 4 결과를 그대로 이어받는다.
- 검증 안전장치(라운드 10): 학습 생존율은 90.0%였지만 검증 생존율은 85.0%로 5.0포인트 내렸고, 검증 분할의 평균 기업 점수도 \$0.184M 줄었다. 학습 성적이 검증으로 이어지지 않자 게이트는 변이를 거부했고 루프는 라운드 9 그래프로 끝났다.

8.3 토폴로지가 걸어온 길

라운드별 점수 뒤에는 그래프 모양의 단계적 변화가 있다.

- 0라운드(초기화): 구조적 사전 없이 Start와 End만 있는 백지 상태다. 중간 절차 안내가 없으므로 LLM이 실행 궤적만으로 행동 순서를 추론해야 하고, 비용이 크고 도구 호출이 겹치며 현금 예측 같은 과업을 놓친다.
- 세대 A(라운드 1, 백본 발견): Start → Month_Start → check_cash_in_bank → cash_flow_forecast_calculation → save_note → check_market_data → Decide_Capital 순서가 제안된다. 자금 결정 전에 현금을 감사하고, 런웨이를 예측하고, 맥락을 저장하고, 시장 가치를 확인하도록 이끈다.
- 세대 B(라운드 2~6, 작업 기억): recall_notes가 Month_Start 직후에 삽입된다. 예측 뒤에 save_note, 다음 달 시작에 recall_notes를 쓰게 하여 핵심 지표를 월 사이에 보존하는 외부 작업 기억이 만들어진다.
- 세대 C(라운드 7, 분지 가지치기): pass_action 노드와 그에 붙은 에지가 잘려 나가고, Decide_Capital에서 fund_raising_request와 book_closing으로 가는 분지만 남는다.
- 세대 D(라운드 8~10, 행정 우회): fund_raising_request에서 book_closing으로 가던 에지가 End로 향하는 직접 링크로 바뀐다. 환경 어댑터가 이미 조달 요청 뒤에 월을 넘겨 주므로, 요청 직후에 또 월을 넘기기보다 절차를 끝내고 새 달의 상태를 다시 평가하라는 안내가 된다.

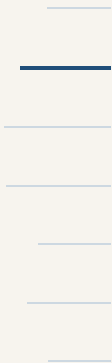
이 루프가 CFO 환경에만 통하는 물건인지 확인하는 교차 검증도 부록에 붙어 있다. 원문 부록 E.4는 HotpotQA와 MultiChallenge에서도 같은 자기진화 절차가 성능을 끌어올림을 보여 주어, 발견이 특정 환경의 우연으로 끝나지 않았음을 확인한다.

참고: 원문 §5.4 (pp.9-10)·부록 E (pp.31-34)

DAY

09

9. 효율성 분석



가이드스 메커니즘에는 두 설계 축이 있다. 첫 축은 에이전트가 그래프의 어느 부분을 보게 할 것인가다. 전체 그래프를 통째로 보여 줄 것인가, 에이전트의 현재 위치에서 국소 서브그래프만 보여 줄 것인가가 그것이다. 둘째 축은 그 부분을 어떻게 소비하게 할 것인가다. 원문을 프롬프트에 그대로 주입할 것인가, 가이드스 모델이 문맥에 맞게 다시 생성하게 할 것인가가 갈림길이다. 두 축은 독립이 아니라 맞물린다. 무엇을 보여 주느냐가 소비 비용을 정하고, 어떻게 소비하느냐가 보여준 내용이 실제 행동으로 옮겨지는 정도를 정하기 때문이다.

원문 Table 3은 이 두 축이 만드는 네 개 구성을 나란히 세운다. 그래프 없음, 전체 그래프 원문 주입, 전체 그래프 생성형, 그리고 서브그래프 생성형(저자들 방법)이다. 모든 구성은 Gemini 3.5 Flash와 공유 솔버 프롬프트 템플릿으로 돌고, 세 개 PG 구성은 동일한 그래프 위에서 움직인다(원문 부록 B.5). 그래프가 같으므로 행 간 차이는 그래프의 내용이 아니라 그래프를 전달하는 방식에서 나온다고 볼 수 있다. 전달 방식의 차이는 이렇다. 원문 주입은 그래프 텍스트를 솔버 프롬프트에 그대로 엮는다. 생성형은 그래프를 가이드스 모델이 먼저 읽고 상황에 맞는 지침으로 바꾼 뒤 건네며, 그 대신 변환을 위한 호출이 하나 더 들어간다.

표 9-1은 원문 Table 3을 그대로 옮긴 것이다. 정확도·루브릭·성공률은 높을수록 좋고, 평균 토큰과 평균 스텝은 낮을수록 좋다.

표 9-1 그래프 구성별 성능과 비용(원문 Table 3 전제, Gemini 3.5 Flash) — MC=MultiChallenge, GDV=GDPval, ALF=ALFWorld. 토큰·스텝은 샘플당 평균이다.

그래프 구성	MC 정 확도	토큰	스텝	GDV 루브릭	토큰	스텝	ALF 성 공률	토큰	스텝
베이스라인(그래프 없음)	80.27	6,629	3.87	54.80	275,638	28.20	72.58	18,055	21.84
전체 그래프, 원문 주입	86.60	10,164	4.54	57.17	264,680	33.55	70.34	21,062	25.00
전체 그래프, 생성형	87.35	14,434	3.08	56.75	448,972	22.07	54.48	96,360	30.05
서브그래프, 생성형(저자들 방법)	89.31	12,295	4.22	63.99	367,738	18.57	81.53	28,064	18.80

행을 따라 내려가면 해석이 저절로 정리된다. 전체 그래프 원문 주입은 구조화 대화에서는 이득이다. MultiChallenge 정확도가 80.27에서 86.60으로 오른다. 그러나 구현 실행에서는 손해다. ALFWorld 성공률이 72.58에서 70.34로 내려간다. 전체 그래프 생성형은 여기서 더 나빠진다. ALFWorld를 54.48까지 떨어뜨리면서 토큰 소모마저 늘린다. 통그래프를 매 스텝 컨텍스트에 엮는 방식은 과업과 상관없는 절차까지 함께 보여 주므로, 환경과 상호작용하며 즉석에서 판단해야 하는 실행 과업에서는 오히려 판단을 흐린다는 것이 이 두 행이 말하는 바다. 원문도 이 결과를 에이전트의 국소 그래프 이웃에 뿌리를 둔 가이드스가 통그래프에서 뽑아 낸 가이드스보다 낫다는 근거로 읽는다.

스텝 열도 같은 방향을 가리킨다. 전체 그래프 원문 주입은 GDPval과 ALFWorld에서 궤적을 늘린다. 솔버 스텝이 각각 28.20에서 33.55로, 21.84에서 25.00으로 불어난다. 구조를 통째로 보여 준다고 행동이 좋아지는 것이 아니라, 길을 잃은 에이전트가 더 많은 스텝을 헤매게 된다는 뜻이다. 보여 주는 정보의 양과 에이전트의 판단력이 비례하지 않는다는 사실을 숫자가 직접 증언한다.

같은 그래프를 놓고 비교하면 국소 생성형 구성이 세 벤치마크 전부에서 최고다. MultiChallenge 89.31, GDPval 63.99, ALFWorld 81.53. 각 설정의 최고 대안, 그래프 없는 베이스라인까지 포함해서, 2.0·6.8·9.0포인트씩 앞선다. 특히 ALFWorld에서의 9.0포인트는 베이스라인을 72.58에서 81.53으로 끌어올린 수치다. 그래프가 있으되 그중 지금 자리와 관련된 부분만, 그것도 생성형으로 전달될 때 절차 구조의 이득이 가장 크게 실현된다.

토큰 절감 폭도 만만치 않다. 전체 그래프 생성형을 기준으로 하면 국소 생성형은 ALFWorld에서 70.9%, GDPval에서 18.1%, MultiChallenge에서 14.8%의 토큰을 덜 쓴다. 세 벤치마크 전부에서 절감이 확인된다. GDPval과 ALFWorld에서는 궤적 자체가 짧아진다. 솔버 스텝이 GDPval에서 28.20에서 18.57로, ALFWorld에서 21.84에서 18.80으로 줄어든다. 정확도를 높이면서 궤적을 동시에 줄였으므로 이득이 스텝 절약에서만 나온 것은 아니다.

그러나 대가도 표에 그대로 남아 있다. 가이드스를 뽑아 내는 호출이 추가되므로 그래프 없는 베이스라인 대비 총 토큰은 GDPval에서 33.4%, ALFWorld에서 55.4% 더 든다. 가이드스가 스텝을 줄여 절약한 것보다 호출 비용이 이 두 과업에

서는 더 크다. 정확도와 궤적 길이의 개선을 토큰 비용의 증가로 사는 구조다. 저자들은 이를 미래 과제로 명시한다. 가이드선을 여러 스텝에 걸쳐 재사용하는 방법과, 필요할 때만 가이드선을 생성하는 선택적 생성 방식이 남은 과제다. 호출당 비용을 여러 스텝에 분산하거나 호출 횟수 자체를 줄이면 정확도 이득을 지키면서 비용 쪽 대가를 깎을 수 있다는 계산이다.

독자가 가져갈 실무적 함의는 단순하다. 에이전트에게 구조를 줄 때는 양보다 위치가 문제다. 에이전트가 지금 서 있는 절차의 이웃만 정확하게 전달하는 것이 통그래프를 통째로 보여 주는 것보다 세 벤치마크에서 모두 나왔다. 구조화 대화처럼 절차 자체가 과업인 곳에서는 그래프를 아예 안 주는 것보다 주는 쪽이 낫고, 구현 실행처럼 즉석 판단이 지배하는 곳에서는 어떻게 주느냐가 성패를 가른다. 같은 그래프라도 전달 설계에 따라 성능과 비용이 그만큼 벌어진다는 사실이 이 표의 가장 실용적인 교훈이다.

참고: 원문 §5.5 (p.10)

DAY

10

10. 결론과 실무 시사점



원문 저자들이 책을 맺으며 되돌아보는 주장은 압축적이다. 절차 그래프(Procedural Graph, PG)는 절차 지식의 명시적이고 편집 가능한 표현이며, 언어모델 에이전트에게 “다음에 무엇을 할까”라는 질문에 조회 가능한 답을 돌려준다. 조회 가능하다는 말의 무게는 가볍지 않다. 에이전트가 지금 어디쯤 와 있는지에 따라 필요한 답이 달라지는데, 그래프는 현재 진행 상황을 관련 전이와 실행 조언에 연결해 알맞은 답을 골라 건네준다. 연결하되, 추론의 유연성은 그대로 지킨다. 그래프가 행동을 명령하지 않고 방향을 건네는 이유가 여기에 있다.

성능 면에서 결과도 뚜렷하다. 과업과 모델 계열을 넘나드는 전체 실험에서 절차 그래프는 메모리 기반 베이스라인 대비 일관된 이득을 냈다. 요약문을 프롬프트 앞에 붙이는 방식, 지난 궤적을 유사도로 꺼내 예시로 보여 주는 방식, 성공과 실패를 대조해 뽑은 통찰을 시스템 프롬프트에 주입하는 방식과 겨뤘을 때에도 순위가 뒤집히지 않았다. 다중 홉 질의응답에서 다중 턴 대화, 도구 호출, 장기 금융 시뮬레이션까지 과업이 바뀌어도 같은 그림이 나왔고, Claude Sonnet 4.6과 Gemini 계열, Grok 계열로 모델을 바꾸어도 그림은 유지되었다. 저장하는 것은 같은 실행 경험이라도 그것을 어떤 구조로 두느냐가 성능을 가른다는 사실이 반복적으로 확인된 셈이다.

자기진화 루프 역시 결론의 한 축을 맡는다. 최소한의 초기화에서 출발해도 효과적인 그래프를 만들어 내며, 처음에는 성능을 오히려 갉아먹던 전문가 사전까지 수리한다. 사람이 설계한 그래프라는 이유로 믿으면 안 되고, 백지라는 이유로 포기할 필요도 없다는 뜻이다. 시작점보다 중요한 것은 시작 이후인 셈이다. 이 두 결과를 겹쳐 보면 결론의 무게가 달라진다. 모델 가중치를 갱신하지 않고도 실행 피드백만으로 절차 지식을 배우고 고칠 길이 열린다는 것이다. 에이전트의 기술을 파인튜닝 비용 없이 모델 바깥 구조에 쌓는다는 발상이, 원문이 실험 전체를 통해 증명하려 한 바로 그 가능성이다.

원문은 이 장면에서 한계도 숨기지 않는다. 가이드نس는 솔버 스텝 수를 줄이면서도 전체 토큰 사용량은 늘린다. 궤적이 짧아진다고 실행 비용이 덜 드는 것은 아니라는 뜻이다. 저자들은 스텝 사이에 가이드نس를 재사용하거나, 필요한 시점에만 만들어 내는 선택적 생성을 미래 과제로 꼽는다. 학습된 절차를 다른 솔버와 도구 인터페이스로 옮겨도 얼마나 잘 통하는지 평가하는 일 역시 남아 있다. 이 두 과제가 채워져야 배운 절차가 특정 구현을 넘어 얼마나 널리 재활용될 수 있는지 명확해진다.

결론 한 장을 한 문장으로 눌러 담으면 이렇다. 절차 지식은 모델 안이나 프롬프트 찌꺼기에 묻어 두는 것이 아니라, 조회하고 고칠 수 있는 구조로 밖에 꺼내 두는 것이 에이전트를 단단하게 만든다. 이어지는 절은 이 결론을 독자의 작업대로 옮겨 놓는 일을 맡는다.

이 책을 읽은 실무자를 위한 시사점

이 절부터는 원문 저자의 결론이 아니라 이 책 번역자의 해설이다. 실험 결과를 자기 프로젝트에 옮겨 붙이려는 독자를 위해 원문 전체를 관통한 설계 선택을 다섯 가지로 묶는다.

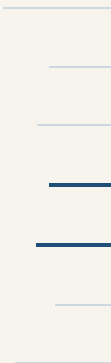
- 절차 지식을 프롬프트 로그에 섞지 말고, 모델 가중치 밖의 조회 가능한 구조로 두라. 가중치에 녹인 기술은 고치려면 재학습이 필요하지만, 그래프에 둔 기술은 노드와 에지를 고쳐 바로 반영된다.
- 가이드نس는 강제가 아니라 편향이어야 에이전트 유연성이 살아난다. 솔버의 출력 형식을 잠가 버리는 대신 다음 스텝의 방향과 함정을 문맥으로 건네는 소프트 통합이, 절차 준수와 추론 자유를 함께 지켰다.
- 학습 루프에는 캐시된 검증 점수 게이트와 거부 메모리라는 안전장치가 성공의 절반이다. 검증 점수를 지지 못한 후보는 채택하지 않고, 그 실패 자체를 다음 정제기 호출에 되먹이는 구조가 있어야 진화가 성능을 지킨다.
- 전문가 사전은 무조건 이득이 아니다. 결함 있는 사전은 그대로 두면 성능을 끌어내리고, 검증 게이트와 함께여야 회복된다.
- 국소 문맥이 전체 주입을 이겼다. 2홉 서브그래프 주입이 전체 그래프 주입보다 성능과 비용 모두에서 유리했다. 절차 지식을 담은 구조가 커질수록 넣을 때마다 드는 비용도 커지므로, 큰 그래프를 통째로 프롬프트에 넣는 습관부터 의심해 볼 일이다.

다섯 가지를 한 줄로 압축하면 이렇다. 지식은 바깥에, 가이드نس는 부드럽게, 진화는 게이트 뒤에서. 절차 지식을 다루는 시스템을 처음 세울 때 이 세 문장이 설계 검토의 출발점이 되기를 바란다.

참고: 원문 §6 (p.11)

DAY

부록 — 프롬프트와 알고리즘



이 부록은 본문의 주장을 재현하려는 독자를 위해 원문이 공개한 프롬프트 세 계열과 자기진화 알고리즘, 실행 사례 두 건을 정리한다. 프롬프트는 ReAct 루프를 지배하는 솔버 실행 프롬프트, 각 스텝에 그래프를 상황 가이드로 바꾸는 가이드스 생성 프롬프트, 오프라인 자기진화를 몰고 가는 정제기 프롬프트다. 템플릿은 모든 벤치마크에서 공유되며 달라지는 것은 도구 목록과 과업 설명뿐이다. 종괄호는 실행 시점에 채워지는 자리표시자다.

| 솔버 실행 프롬프트

솔버 실행 프롬프트는 ReAct 루프의 규율을 잡는다. 필요한 인자는 시스템 프롬프트 `system_prompt`, 그래프에서 만든 가이드스 `procedural_graph_guidance`, 지금까지의 궤적 `trajectory` 세 가지다. 출력 형식은 Thought와 Action을 한 쌍으로, 딱 한 번 요구한다. 환경의 응답을 대신 써 내는 것을 이중으로 금지하는 이유는, 모델이 관측을 상상해 궤적을 오염시키는 실패가 흔하기 때문이다.

```
{system_prompt}
Procedural Graph Guidance: {procedural_graph_guidance}

You must interleave Thought and Action. Output exactly:
Thought: <다음에 무엇을 할지에 대한 추론>
Action: <tool_name>(arg1=val1, arg2=val2, ...)

# Observation 블록을 쓰거나 이후 스텝을 훑내 내지 않는다.
# Thought 하나와 Action 하나만 출력한다.
Current Trajectory: {trajectory}
Thought:
```

| 가이드스 생성 프롬프트

가이드스 생성 프롬프트는 그래프 문맥을 상황 가이드스로 번역한다. 기본 변형은 국소 서브그래프를 받는다. 인자는 과업 설명 `task_description`, 그래프 문맥 설명과 직렬화된 그래프 `graph_context_desc` `subgraph_summary`, 현재 질의나 관측 `query`, 최근 실행 창 `recent_context`, 그래프 출처 표기 `graph_source`다. 지시문은 예지가 실은 `condition` `guidance` `pitfalls` 속성을 근거로, 다음 스텝과 피해야 할 함정, 최근 실패의 회복법을 담은 구체적 가이드스를 요구하고, 그래프 문맥에 정의된 명령 패턴, 경로, 도구, 인자는 관련되는 한 반드시 포함하라고 못 박는다.

전체 그래프 변형은 이 프롬프트와 문자 수준에서 동일하다. 다른 것은 그래프 문맥 솔라에 국소 서브그래프 대신 전체 그래프가 들어가는 것뿐이어서, 본문의 비교 실험 두 행은 프롬프트 문구나 요구 출력 길이가 아니라 그래프 범위만 다른 셈이다. 직렬화된 그래프 문맥은 활성 노드 헤더에 노드 ID와 유형, 설명을 담고, 전이들은 홑별로 묶여 각자 `condition`, `guidance`, `pitfalls`을 따라 온다.

```
You are an expert cognitive architect and execution guide
for an AI agent solving the task: {task_description}
Here is {graph_context_desc}: {subgraph_summary}
Here is the current active query / observation: {query}
Here is the agent's recent execution trajectory: {recent_context}

# 예지 속성 condition / guidance / pitfalls을 근거로
# 다음 스텝, 피할 함정, 최근 실패의 회복법을 담아 가이드스를 생성한다.
# 그래프 문맥에 정의된 명령 패턴·경로·도구·인자는 관련 시 반드시 포함한다.
```

| 오프라인 정제기 프롬프트

오프라인 정제기 프롬프트는 자기진화 루프의 손이다. 인자로 과업 설명 `task_description`, 정제 모드 `mode`, 실행 가능 도구 목록 `available_tools_list`, 최근 실행 궤적들 `attempts_block`, 현재 그래프의 직렬화 표현 `current_graph_json`, 이전에 기각된 후보들 `rejected_block`을 받는다. 전문가 사전에서 시작하는 정적 모드는 루프와 교착, 실패로 이어지는

노드와 에지를 가지치기하고 빠진 것을 보태라 이르며, 백지 모드는 그래프가 출발과 종결뿐일 때 궤적 속 성공 패턴에서 완전한 새 그래프를 합성하라고 요구한다.

규칙 일곱 개가 편집의 자격을 검사한다. 행동 노드는 도구 목록 이름과 일치해야 하고, 조건에는 전이가 발화하는 자연어 전제를 적으며, 새로 더하는 모든 에지에 guidance 문자열을 반드시 첨부해야 한다. pitfalls은 성급한 행동과 금지어, 형식 실수를 경고하고, 나머지 규칙은 과적합 방지와 기존 노드 ID 보존, 주기 정책과 종결 노드 도달성을 다룬다. 출력은 `add_nodes delete_nodes add_edges delete_edges` 네 배열을 담은 단일 블록이다. `delete_edges` 항목 하나는 관계와 무관하게 같은 양끝의 모든 에지를 지우므로, 남겨 둘 전이는 나중에 적용되는 `add_edges`에 넣어야 한다.

```
You are an expert cognitive architect optimizing a Procedural Graph.
Task context: {task_description}
Refinement mode: {mode} # static / scratch x onetime / incremental
Available Tool Actions: {available_tools_list} # 에이전트가 실행할 수 있는 유일한 행동 목록
Recent execution trajectories: {attempts_block}
Current Procedural Graph representation: {current_graph_json}
Previously rejected candidates: {rejected_block}

# 규칙 1: ACTION 노드는 도구 목록의 이름과 일치해야 한다.
# 규칙 2-4: 에지마다 condition(전제), guidance(새 에지엔 필수), pitfalls을 채운다.
# 규칙 5-7: 과적합 방지, 기존 노드 ID 보존, 주기 정책과 종결 노드 도달성.
Output (single block only):
{ "add_nodes": [{"id":..., "type":"ACTION", "description":...}],
  "delete_nodes": ["node_id"],
  "add_edges": [{"source":..., "target":..., "relation":...,
                  "condition":..., "guidance":..., "pitfalls":...}],
  "delete_edges": [{"source":..., "target":...}] }
```

자기진화 알고리즘

원문의 Algorithm 1은 오프라인 페루프를 한 눈에 보여 준다. S_k 는 그래프 G_k 에 캐시된 검증 점수이고, c 는 주기를 허용할지 정하는 정책이다. 궤적 한도 L_{max} 를 적용하는 Tail 연산은 입력 앞부분을 잘라 끝부분을 보존하므로 짧은 입력은 그대로 지나간다. 그래프는 각 학습·검증 에피소드 동안 고정되며, `SerializeRejections`는 거부 기록에서 후보 그래프와 검증 점수, 구조 결함 진단을 꺼내 정제기에 넘긴다.

```
# Algorithm 1: Offline Closed-Loop Procedural Graph Self-Evolution
# 입력: 초기 그래프  $G_0$ , 학습/검증 집합  $D_{train} \cdot D_{val}$ , 라운드 예산  $K$ ,
#       궤적 한도  $L_{max}$ , 주기 정책  $c$  | 출력: 진화된 그래프  $G_K$ 
 $S_0 \leftarrow \text{Evaluate}(G_0, D_{val})$ ;  $H_{rejected} \leftarrow []$ 
for  $k = 1 \dots K$ :
  # 1단계 - 유지: 승인이 없으면 직전 상태를 그대로 보존한다
   $G_k \leftarrow G(k-1)$ ;  $S_k \leftarrow S(k-1)$ 
  # 2단계 - 굴러기와 문맥 직렬화
   $E_k \leftarrow \text{Rollout}(G(k-1), B_k)$  # 학습 배치  $B_k$ 의 궤적과 점수
   $C_k \leftarrow \text{Tail}_{L_{max}}(\text{ConcatTrajectories}(E_k))$  # 앞부분을 잘라 끝을 보존
   $R_k \leftarrow \text{SerializeRejections}(H_{rejected})$  # 기각 후보의 점수·구조 진단
  # 3단계 - 정제기 호출과 후보 준비
   $dG_k \leftarrow \text{Refiner}(G(k-1), C_k, \{S(k)_i\}, R_k)$ 
   $(G_{cand\_k}, dk) \leftarrow \text{PrepareCandidate}(G(k-1), dG_k, c)$ 
  if  $dk \neq \{\}$ : # 구조 결함 - 검증 굴러기 없이 기각 기록만 남긴다
    append  $(dG_k, G_{cand\_k}, E_k, dk)$  to  $H_{rejected}$ ; continue
  # 4단계 - 검증 게이트: 타이까지 포함해 지지 않으면 채택하지 않는다
   $Scand\_k \leftarrow \text{Evaluate}(G_{cand\_k}, D_{val})$ 
  if  $Scand\_k \geq S(k-1)$ :  $G_k \leftarrow G_{cand\_k}$ ;  $S_k \leftarrow Scand\_k$ 
  else: append  $(dG_k, G_{cand\_k}, E_k, Scand\_k)$  to  $H_{rejected}$ 
return  $G_K$ 
```

후보 준비 단계의 세부도 알고리즘 설명에 붙어 있다. `PrepareCandidate`는 유지 중인 그래프의 복사본에 편집을 적용하되 노드와 에지를 더하기 전에 지우고, 형식이 틀린 편집과 유효하지 않은 노드·관계 유형, 끝점이 빠진 에지를 결함으로 보

고한다. 주기를 금지하면 주기를 닫는 에지를 검증 전에 걷어내고, 허용하면 이 수선과 비순환 검사를 건너뛴다. 남은 검사는 에지 끝점의 유효성과 종결 노드 도달성이며, 행동 노드 이름과 도구 목록의 대응은 정제기 프롬프트의 요구로 남아 있다.

BFCL에서 요청된 견적 이후 정지

원문 부록 F.1의 사례다. Gemini 3.5 Flash로 돌린 BFCL 검사 샘플 051에서 두 에이전트는 같은 요청, 이코노미석 항공권 견적 조회를 받는다. `get_flight_cost`가 돌려준 `travel_cost_list`는 `[220.0]`이다.

여기서 두 에이전트의 길이 같린다. 한 번의 오프라인 갱신으로 백지에서 만든 그래프를 쓰는 PG 에이전트는 가격을 보고하고 그대로 턴을 끝낸다. 솔버 궤적에 남은 근거 문장은 “Since the user did not explicitly request to book the flight, I should not proceed to book_flight.”이며, 실제 행동은 Rivermist에서 Stonebrook까지 10월 6일 이코노미석 항공 값 \$220.00을 최종 응답으로 돌려주는 Finish였다. 베이스라인은 인증과 카드 조작, `book_flight`로 나아가고, 예약이 실패하자 예산 한도를 바꿔 예약을 다시 시도한다. 환경은 0턴에서 상태 불일치를 보고한다. 요청된 것 이상의 행동이 요청을 망친 셈이다. PG 실행은 다음 사용자 지시로 넘어가 성공한다. 불필요한 추가 행동을 가른 것은 그래프가 준 정지 조건이었다.

MultiChallenge에서 응답 목표 교정

원문 부록 F.2의 사례다. MultiChallenge 검증 샘플 059는 수동태만 쓰라는 앞선 지시 아래 재생에너지나 셰필드에 관한 농담을 요구하고, 환경은 평가 질문 “Did the model consistently use passive sentence construction?”을 함께 노출한다. 구축 모드 3의 2세대 후보와 3세대 후보 평가는 초기 질의와 대화 이력이 완전히 같다.

2세대 후보는 `AnalyzeTargetQuestion`에서 Finish로 가는 에지에 “Direct finish for simple evaluations without drafting.”라는 guidance를 달고 있었다. 기록된 행동 순서는 `ParseHistory`에서 `AnalyzeTargetQuestion`, 이어 Finish였고, 최종 응답은 “No, the model did not consistently use passive sentence construction.”이었다. 농담을 만들어야 할 에이전트가 대화를 평가하는 쪽으로 목표가 미끄러진 것이다. 결함 있는 전문가 사전이 유독한 잘못된 응답 목표가 그대로 드러난 장면이다.

3세대 그래프의 수정은 정확히 이 지점을 노린다. Finish로 가는 직통 에지가 삭제되고, `ExtractConstraints`에서 Finish로 가는 에지의 guidance는 메타 평가를 쓰거나 평가 질문에 직접 답하지 말라는 문장으로 바뀐다. 행동 순서에는 `ExtractConstraints`가 끼어 `ParseHistory`, `AnalyzeTargetQuestion`, `ExtractConstraints`, Finish가 된다. 최종 응답은 “A joke about renewable energy is being shared. Why are wind turbines loved by everyone? Many fans are known to be made by them.”으로, 수동태 농담이라는 과업 목표에 맞는 산출물이다. 성공 지표가 0에서 1로 바뀌었고, 바뀐 경로와 에지 조언은 대화를 평가하던 응답에서 사용자에게 답하는 응답으로의 전환을 그대로 반영한다.

참고: 원문 부록 B.5–B.6 (pp.21–24)·부록 F (pp.35–36)